

In the alternative view, a synchronous primitive is one in which the sender is blocked until the receiver has accepted the message and the acknowledgement has gotten back to the sender. Everything else is asynchronous in this view. There is complete agreement that if the sender gets control back before the message has been copied or sent, the primitive is asynchronous. Similarly, everyone agrees that when the sender is blocked until the receiver has acknowledged the message, we have a synchronous primitive.

The disagreement comes on whether the intermediate cases (message copied or copied and sent, but not acknowledged) counts as one or the other. Operating systems designers tend to prefer our way, since their concern is with buffer management and message transmission. Programming language designers tend to prefer the alternative definition, because that is what counts at the language level.

Just as *send* can be blocking or nonblocking, so can *receive*. A nonblocking *receive* just tells the kernel where the buffer is, and returns control almost immediately. Again here, how does the caller know when the operation has completed? One way is to provide an explicit *wait* primitive that allows the receiver to block when it wants to. Alternatively (or in addition to *wait*), the designers may provide a *test* primitive to allow the receiver to poll the kernel to check on the status. A variant on this idea is a *conditional_receive*, which either gets a message or signals failure, but in any event returns immediately, or within some timeout interval. Finally, here too, interrupts can be used to signal completion. For the most part, a blocking version of *receive* is much simpler and greatly preferred.

If multiple threads of control are present within a single address space, the arrival of a message can cause a thread to be created spontaneously. We will come back to this issue after we have looked at threads in Chap. 4.

An issue closely related to blocking versus nonblocking calls is that of timeouts. In a system in which *send* calls block, if there is no reply, the sender will block forever. To prevent this situation, in some systems the caller may specify a time interval within which it expects a reply. If none arrives in that interval, the *send* call terminates with an error status.

2.3.5. Buffered versus Unbuffered Primitives

Just as system designers have a choice between blocking and nonblocking primitives, they also have a choice between buffered and unbuffered primitives. The primitives we have described so far are essentially **unbuffered primitives**. What this means is that an address refers to a specific process, as in Fig. 2-9. A call *receive(addr, &m)* tells the kernel of the machine on which it is running that the calling process is listening to address *addr* and is prepared to receive one

message sent to that address. A single message buffer, pointed to by m , is provided to hold the incoming message. When the message comes in, the receiving kernel copies it to the buffer and unblocks the receiving process. The use of an address to refer to a specific process is illustrated in Fig. 2-12(a).

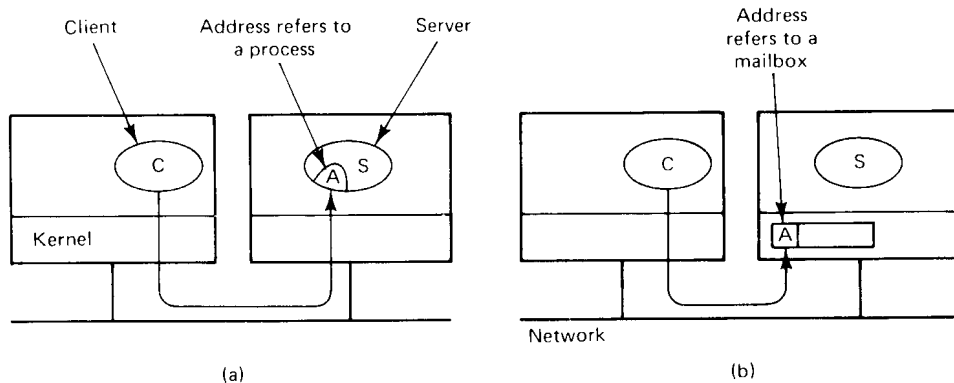


Fig. 2-12. (a) Unbuffered message passing. (b) Buffered message passing.

This scheme works fine as long as the server calls *receive* before the client calls *send*. The call to *receive* is the mechanism that tells the server's kernel which address the server is using and where to put the incoming message. The problem arises when the *send* is done before the *receive*. How does the server's kernel know which of its processes (if any) is using the address in the newly arrived message, and how does it know where to copy the message? The answer is simple: it does not.

One implementation strategy is to just discard the message, let the client time out, and hope the server has called *receive* before the client retransmits. This approach is easy to implement, but with bad luck, the client (or more likely, the client's kernel) may have to try several times before succeeding. Worse yet, if enough consecutive attempts fail, the client's kernel may give up, falsely concluding that the server has crashed or that the address is invalid.

In a similar vein, suppose that two or more clients are using the server of Fig. 2-9(a). After the server has accepted a message from one of them, it is no longer listening to its address until it has finished its work and gone back to the top of the loop to call *receive* again. If it takes a while to do the work, the other clients may make multiple attempts to send to it, and some of them may give up, depending on the values of their retransmission timers and how impatient they are.

The second approach to dealing with this problem is to have the receiving kernel keep incoming messages around for a little while, just in case an appropriate *receive* is done shortly. Whenever an "unwanted" message arrives,

a timer is started. If the timer expires before a suitable *receive* happens, the message is discarded.

Although this method reduces the chance that a message will have to be thrown away, it introduces the problem of storing and managing prematurely arriving messages. Buffers are needed and have to be allocated, freed, and generally managed. A conceptually simple way of dealing with this buffer management is to define a new data structure called a **mailbox**. A process that is interested in receiving messages tells the kernel to create a mailbox for it, and specifies an address to look for in network packets. Henceforth, all incoming messages with that address are put in the mailbox. The call to *receive* now just removes one message from the mailbox, or blocks (assuming blocking primitives) if none is present. In this way, the kernel knows what to do with incoming messages and has a place to put them. This technique is frequently referred to as a **buffered primitive**, and is illustrated in Fig. 2-12(b).

At first glance, mailboxes appear to eliminate the race conditions caused by messages being discarded and clients giving up. However, mailboxes are finite and can fill up. When a message arrives for a mailbox that is full, the kernel once again is confronted with the choice of either keeping it around for a while, hoping that at least one message will be extracted from the mailbox in time, or discarding it. These are precisely the same choices we had in the unbuffered case. Although we have perhaps reduced the probability of trouble, we have not eliminated it, and have not even managed to change its nature.

In some systems, another option is available: do not let a process send a message if there is no room to store it at the destination. To make this scheme work, the sender must block until an acknowledgement comes back saying that the message has been received. If the mailbox is full, the sender can be backed up and retroactively suspended as though the scheduler had decided to suspend it just *before* it tried to send the message. When space becomes available in the mailbox, the sender is allowed to try again.

2.3.6. Reliable versus Unreliable Primitives

So far we have tacitly assumed that when a client sends a message, the server will receive it. As usual, reality is more complicated than our abstract model. Messages can get lost, which affects the semantics of the message passing model. Suppose that blocking primitives are being used. When a client sends a message, it is suspended until the message has been sent. However, when it is restarted, there is no guarantee that the message has been delivered. The message might have been lost.

Three different approaches to this problem are possible. The first one is just to redefine the semantics of *send* to be unreliable. The system gives no guarantee about messages being delivered. Implementing reliable communication is

entirely up to the users. The post office works this way. When you drop a letter in a letterbox, the post office does its best (more or less) to deliver it, but it promises nothing.

The second approach is to require the kernel on the receiving machine to send an acknowledgement back to the kernel on the sending machine. Only when this acknowledgement is received will the sending kernel free the user (client) process. The acknowledgement goes from kernel to kernel; neither the client nor the server ever sees an acknowledgement. Just as the request from client to server is acknowledged by the server's kernel, the reply from the server back to the client is acknowledged by the client's kernel. Thus a request and reply now take four messages, as shown in Fig. 2-13(a).

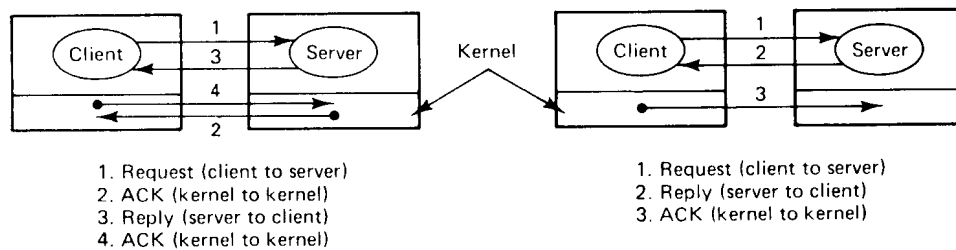


Fig. 2-13. (a) Individually acknowledged messages. (b) Reply being used as the acknowledgement of the request. Note that the ACKs are handled entirely within the kernels.

The third approach is to take advantage of the fact that client-server communication is structured as a request from the client to the server followed by a reply from the server to the client. In this method, the client is blocked after sending a message. The server's kernel does not send back an acknowledgement. Instead, the reply itself acts as the acknowledgement. Thus the sender remains blocked until the reply comes in. If it takes too long, the sending kernel can resend the request to guard against the possibility of a lost message. This approach is shown in Fig. 2-13(b).

Although the reply functions as an acknowledgement for the request, there is no acknowledgement for the reply. Whether this omission is serious or not depends on the nature of the request. If, for example, the client asks the server to read a block of a file and the reply is lost, the client will just repeat the request and the server will send the block again. No damage is done and little time is lost.

On the other hand, if the request requires extensive computation on the part of the server, it would be a pity to discard the answer before the server is sure that the client has received the reply. For this reason, an acknowledgement from the client's kernel to the server's kernel is sometimes used. Until this packet is

received, the server's *send* does not complete and the server remains blocked (assuming blocking primitives are used). In any event, if the reply is lost and the request is retransmitted, the server's kernel can see that the request is an old one and just send the reply again without waking up the server. Thus in some systems the reply is acknowledged and in others it is not [see Fig. 2-13(b)].

A compromise between Fig. 2-13(a) and Fig. 2-13(b) that often works goes like this. When a request arrives at the server's kernel, a timer is started. If the server sends the reply quickly enough (i.e., before the timer expires), the reply functions as the acknowledgement. If the timer goes off, a separate acknowledgement is sent. Thus in most cases, only two messages are needed, but when a complicated request is being carried out, a third one is used.

2.3.7. Implementing the Client-Server Model

In the preceding sections we have looked at four design issues, addressing, blocking, buffering, and reliability, each with several options. The major alternatives are summarized in Fig. 2-14. For each item we have listed three possibilities. Simple arithmetic shows that there are $3^4 = 81$ combinations. Not all of them are equally good. Nevertheless, just in this one area (message passing), the system designers have a considerable amount of leeway in choosing a set (or multiple sets) of communication primitives.

Item	Option 1	Option 2	Option 3
Addressing	Machine number	Sparse process addresses	ASCII names looked up via server
Blocking	Blocking primitives	Nonblocking with copy to kernel	Nonblocking with interrupt
Buffering	Unbuffered, discarding unexpected messages	Unbuffered, temporarily keeping unexpected messages	Mailboxes
Reliability	Unreliable	Request-Ack-Reply Ack	Request-Reply-Ack

Fig. 2-14. Four design issues for the communication primitives and some of the principal choices available.

While the details of how message passing is implemented depend to some extent on which choices are made, it is still possible to make some general comments about the implementation, protocols, and software. To start with, virtually all networks have a maximum packet size, typically a few thousand bytes at most. Messages larger than this must be split up into multiple packets and sent separately. Some of these packets may be lost or garbled, and they may even

arrive in the wrong order. To deal with this problem, it is usually sufficient to assign a message number to each message, and put it in each packet belonging to the message, along with a sequence number giving the order of the packets.

However, an issue that still must be resolved is the use of acknowledgements. One strategy is to acknowledge each individual packet. Another one is to acknowledge only entire messages. The former has the advantage that if a packet is lost, only that packet has to be retransmitted, but it has the disadvantage of requiring more packets on the network. The latter has the advantage of fewer packets, but the disadvantage of a more complicated recovery when a packet is lost (because a client timeout requires retransmitting the entire message). The choice depends largely on the loss rate of the network being used.

Another interesting issue is the underlying protocol used in client-server communication. Figure 2-15 shows six packet types that are commonly used to implement client-server protocols. The first one is the REQ packet, used to send a request message from a client to a server. (For simplicity, for the rest of this section we will assume that each message fits in a single packet.) The next one is the REP packet that carries results back from the server to the client. Then comes the ACK packet, which is used in reliable protocols to confirm the correct receipt of a previous packet.

Code	Packet type	From	To	Description
REQ	Request	Client	Server	The client wants service
REP	Reply	Server	Client	Reply from the server to the client
ACK	Ack	Either	Other	The previous packet arrived
AYA	Are you alive?	Client	Server	Probe to see if the server has crashed
IAA	I am alive	Server	Client	The server has not crashed
TA	Try again	Server	Client	The server has no room
AU	Address unknown	Server	Client	No process is using this address

Fig. 2-15. Packet types used in client-server protocols.

The next four packet types are not essential, but often useful. Consider the situation in which a request has been sent successfully from the client to the server and the acknowledgement has been received. At this point the client's kernel knows that the server is working on the request. But what happens if no answer is forthcoming within a reasonable time? Is the request really that complicated, or has the server crashed? To be able to distinguish these two cases, the AYA packet is sometimes provided, so the client can ask the server what is going on. If the answer is IAA, the client's kernel knows that all is well and just

continues to wait. Even better is a REP packet, of course. If the AYA does not generate any response, the client's kernel waits a short interval and tries again. If this procedure fails more than a specified number of times, the client's kernel normally gives up and reports failure back to the user. The AYA and IAA packets can also be used even in a protocol in which REQ packets are not acknowledged. They allow the client to check on the server's status.

Finally, we come to the last two packet types, which are useful in case a REQ packet cannot be accepted. There are two reasons why this might happen, and it is important for the client's kernel to be able to distinguish them. One reason is that the mailbox to which the request is addressed is full. By sending this packet back to the client's kernel, the server's kernel can indicate that the address is valid, and the request should be repeated later. The other reason is that the address does not belong to any process or mailbox. Repeating it later will not help.

This situation can also arise when buffering is not used and the server is not currently blocked in a *receive* call. Since having the server's kernel forget that the address even exists in between calls to *receive* can lead to problems, in some systems a server can make a call whose only function is to register a certain address with the kernel. In that way, at least the kernel can tell the difference between an address to which no one is currently listening, and one that is simply wrong. It can then send TA in the former case and AU in the latter.

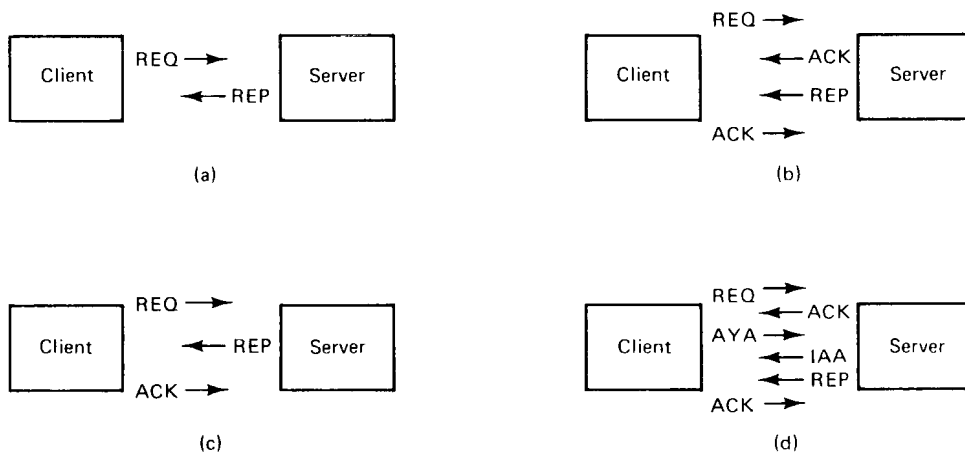


Fig. 2-16. Some examples of packet exchanges for client-server communication.

Many packet sequences are possible. A few common ones are shown in Fig. 2-16. In Fig. 2-16(a), we have the straight request/reply, with no acknowledgement. In Fig. 2-16(b), we have a protocol in which each message is acknowledged individually. In Fig. 2-16(c), we see the reply acting as the

acknowledgement, reducing the sequence to three packets. Finally, in Fig. 2-16(d), we see a nervous client checking to see if the server is still there.

2.4. REMOTE PROCEDURE CALL

Although the client-server model provides a convenient way to structure a distributed operating system, it suffers from one incurable flaw: the basic paradigm around which all communication is built is input/output. The procedures *send* and *receive* are fundamentally engaged in doing I/O. Since I/O is not one of the key concepts of centralized systems, making it the basis for distributed computing has struck many workers in the field as a mistake. Their goal is to make distributed computing look like centralized computing. Building everything around I/O is not the way to do it.

This problem has long been known, but little was done about it until a paper by Birrell and Nelson (1984) introduced a completely different way of attacking the problem. Although the idea is refreshingly simple (once someone has thought of it), the implications are often subtle. In this section we will examine the concept, its implementation, its strengths, and its weaknesses.

In a nutshell, what Birrell and Nelson suggested was allowing programs to call procedures located on other machines. When a process on machine *A* calls a procedure on machine *B*, the calling process on *A* is suspended, and execution of the called procedure takes place on *B*. Information can be transported from the caller to the callee in the parameters and can come back in the procedure result. No message passing or I/O at all is visible to the programmer. This method is known as **remote procedure call**, or often just **RPC**.

While the basic idea sounds simple and elegant, subtle problems exist. To start with, because the calling and called procedures run on different machines, they execute in different address spaces, which causes complications. Parameters and results also have to be passed, which can be complicated, especially if the machines are not identical. Finally, both machines can crash, and each of the possible failures causes different problems. Still, most of these can be dealt with, and RPC is a widely-used technique that underlies many distributed operating systems.

2.4.1. Basic RPC Operation

To understand how RPC works, it is important first to fully understand how a conventional (i.e., single machine) procedure call works. Consider a call like

```
count = read(fd, buf, nbytes);
```

where *fd* is an integer, *buf* is an array of characters, and *nbytes* is another

integer. If the call is made from the main program, the stack will be as shown in Fig. 2-17(a) before the call. To make the call, the caller pushes the parameters onto the stack in order, last one first, as shown in Fig. 2-17(b). (The reason that C compilers push the parameters in reverse order has to do with *printf*—by doing so, *printf* can always locate its first parameter, the format string.) After *read* has finished running, it puts the return value in a register, removes the return address, and transfers control back to the caller. The caller then removes the parameters from the stack, returning it to the original state, as shown in Fig. 2-17(c).

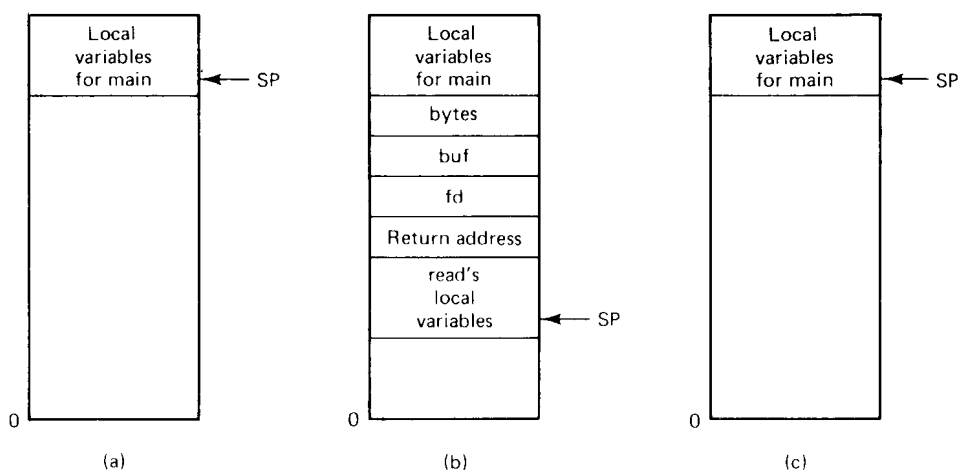


Fig. 2-17. (a) The stack before the call to *read*. (b) The stack while the called procedure is active. (c) The stack after the return to the caller.

Several things are worth noting. For one, in C, parameters can be **call-by-value** or **call-by-reference**. A value parameter, such as *fd* or *nbytes*, is simply copied to the stack as shown in Fig. 2-17(b). To the called procedure, a value parameter is just an initialized local variable. The called procedure may modify it, but such changes do not affect the original value at the calling side.

A reference parameter in C is a pointer to a variable (i.e., the address of the variable), rather than the value of the variable. In the call to *read*, the second parameter is a reference parameter because arrays are always passed by reference in C. What is actually pushed onto the stack is the address of the character array. If the called procedure uses this parameter to store something into the character array, it *does* modify the array in the calling procedure. The difference between call-by-value and call-by-reference is quite important for RPC, as we shall see.

One other parameter passing mechanism also exists, although it is not used

in C. It is called **call-by-copy/restore**. It consists of having the variable copied to the stack by the caller, as in call-by-value, and then copied back after the call, overwriting the caller's original value. Under most conditions, this achieves the same effect as call-by-reference, but in some situations, such as the same parameter being present multiple times in the parameter list, the semantics are different.

The decision of which parameter passing mechanism to use is normally made by the language designers and is a fixed property of the language. Sometimes it depends on the data type being passed. In C, for example, integers and other scalar types are always passed by value, whereas arrays are always passed by reference, as we have seen. In contrast, Pascal programmers can choose which mechanism they want for each parameter. The default is call-by-value, but programmers can force call-by-reference by inserting the keyword **var** before specific parameters. Some Ada[®] compilers use copy/restore for **in out** parameters, but others use call-by-reference. The language definition permits either choice, which makes the semantics a bit fuzzy.

The idea behind RPC is to make a remote procedure call look as much as possible like a local one. In other words, we want RPC to be transparent—the calling procedure should not be aware that the called procedure is executing on a different machine, or vice versa. Suppose that a program needs to read some data from a file. The programmer puts a call to *read* in the code to get the data. In a traditional (single-processor) system, the *read* routine is extracted from the library by the linker and inserted into the object program. It is a short procedure, usually written in assembly language, that puts the parameters in registers and then issues a READ system call by trapping to the kernel. In essence, the *read* procedure is a kind of interface between the user code and the operating system.

Even though *read* issues a kernel trap, it is called in the usual way, by pushing the parameters onto the stack, as shown in Fig. 2-17. Thus the programmer does not know that *read* is actually doing something fishy.

RPC achieves its transparency in an analogous way. When *read* is actually a remote procedure (e.g., one that will run on the file server's machine), a different version of *read*, called a **client stub**, is put into the library. Like the original one, it too, is called using the calling sequence of Fig. 2-17. Also like the original one, it too, traps to the kernel. Only unlike the original one, it does not put the parameters in registers and ask the kernel to give it data. Instead, it packs the parameters into a message and asks the kernel to send the message to the server as illustrated in Fig. 2-18. Following the call to *send*, the client stub calls *receive*, blocking itself until the reply comes back.

When the message arrives at the server, the kernel passes it up to a **server stub** that is bound with the actual server. Typically the server stub will have called *receive* and be blocked waiting for incoming messages. The server stub

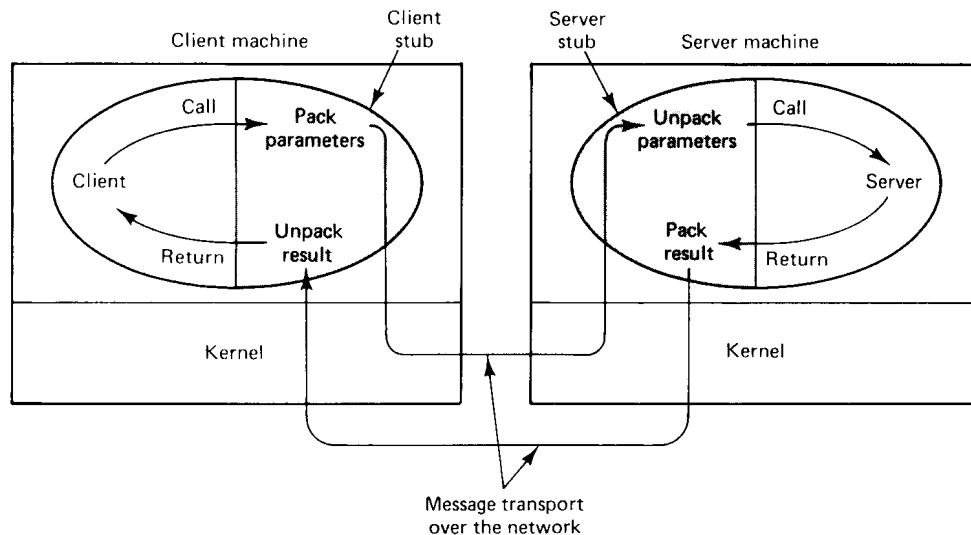


Fig. 2-18. Calls and messages in an RPC. Each ellipse represents a single process, with the shaded portion being the stub.

unpacks the parameters from the message and then calls the server procedure in the usual way (i.e., as in Fig. 2-17). From the server's point of view, it is as though it is being called directly by the client—the parameters and return address are all on the stack where they belong and nothing seems unusual. The server performs its work and then returns the result to the caller in the usual way. For example, in the case of *read*, the server will fill the buffer, pointed to by the second parameter, with the data. This buffer will be internal to the server stub.

When the server stub gets control back after the call has completed, it packs the result (the buffer) in a message and calls *send* to return it to the client. Then it goes back to the top of its own loop to call *receive*, waiting for the next message.

When the message gets back to the client machine, the kernel sees that it is addressed to the client process (to the stub part of that process, but the kernel does not know that). The message is copied to the waiting buffer and the client process unblocked. The client stub inspects the message, unpacks the result, copies it to its caller, and returns in the usual way. When the caller gets control following the call to *read*, all it knows is that its data are available. It has no idea that the work was done remotely instead of by the local kernel.

This blissful ignorance on the part of the client is the beauty of the whole scheme. As far as it is concerned, remote services are accessed by making ordinary (i.e., local) procedure calls, not by calling *send* and *receive* as in Fig. 2-9.

All the details of the message passing are hidden away in the two library procedures, just as the details of actually making system call traps are hidden away in traditional libraries.

To summarize, a remote procedure call occurs in the following steps:

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and traps to the kernel.
3. The kernel sends the message to the remote kernel.
4. The remote kernel gives the message to the server stub.
5. The server stub unpacks the parameters and calls the server.
6. The server does the work and returns the result to the stub.
7. The server stub packs it in a message and traps to the kernel.
8. The remote kernel sends the message to the client's kernel.
9. The client's kernel gives the message to the client stub.
10. The stub unpacks the result and returns to the client.

The net effect of all these steps is to convert the local call by the client procedure to the client stub to a local call to the server procedure without either client or server being aware of the intermediate steps.

2.4.2. Parameter Passing

The function of the client stub is to take its parameters, pack them into a message, and send it to the server stub. While this sounds straightforward, it is not quite as simple as it at first appears. In this section we will look at some of the issues concerned with parameter passing in RPC systems. Packing parameters into a message is called **parameter marshaling**.

As the simplest possible example, consider a remote procedure, $sum(i, j)$, that takes two integer parameters and returns their arithmetic sum. (As a practical matter, one would not normally make such a simple procedure remote due to the overhead, but as an example it will do.) The call to sum , with parameters 4 and 7, is shown in the left-hand portion of the client process in Fig. 2-19. The client stub takes its two parameters and puts them in a message as indicated. It also puts the name or number of the procedure to be called in the message because the server might support several different calls, and it has to be told which one is required.

When the message arrives at the server, the stub examines the message to see which procedure is needed, and then makes the appropriate call. If the

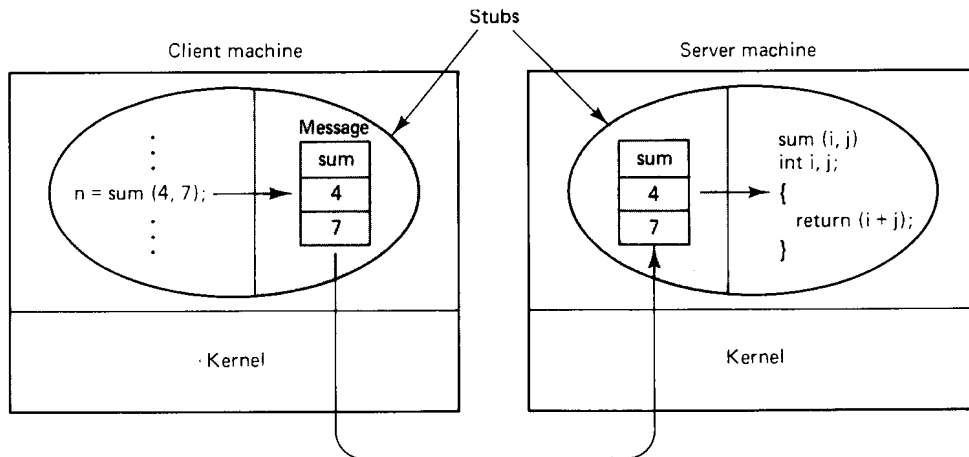


Fig. 2-19. Computing `sum(4, 7)` remotely.

server also supports the remote procedures *difference*, *product*, and *quotient*, the server stub might have a switch statement in it, to select the procedure to be called, depending on the first field of the message. The actual call from the stub to the server looks much like the original client call, except that the parameters are variables initialized from the incoming message, rather than constants.

When the server has finished, the server stub gains control again. It takes the result, provided by the server, and packs it into a message. This message is sent back to the client stub, which unpacks it and returns the value to the client procedure (not shown in the figure).

As long as the client and server machines are identical and all the parameters and results are scalar types, such as integers, characters, and Booleans, this model works fine. However, in a large distributed system, it is common that multiple machine types are present. Each machine often has its own representation for numbers, characters, and other data items. For example, IBM mainframes use the EBCDIC character code, whereas IBM personal computers use ASCII. As a consequence, it is not possible to pass a character parameter from an IBM PC client to an IBM mainframe server using the simple scheme of Fig. 2-19: the server will interpret the character incorrectly.

Similar problems can occur with the representation of integers (1s complement versus 2s complement), and especially with floating-point numbers. In addition, an even more annoying problem exists because some machines, such as the Intel 486, number their bytes from right to left, whereas others, such as the Sun SPARC, number them the other way. The Intel format is called **little**

endian and the SPARC format is called **big endian**, after the politicians in *Gulliver's Travels* who went to war over which end of an egg to break (Cohen, 1981). As an example, consider a server with two parameters, an integer and a four-character string. Each parameter requires one 32-bit word. Figure 2-20(a) shows what the parameter portion of a message built by a client stub on an Intel 486 might look like. The first word contains the integer parameter, 5 in this case, and the second contains the string "JILL".

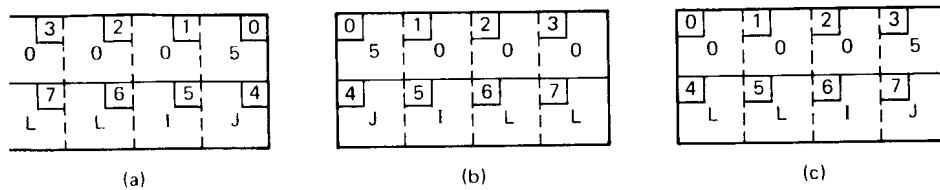


Fig. 2-20. (a) The original message on the 486. (b) The message after receipt on the SPARC. (c) The message after being inverted. The little numbers in boxes indicate the address of each byte.

Since messages are transferred byte for byte (actually, bit for bit) over the network, the first byte sent is the first byte to arrive. In Fig. 2-20(b) we show what the message of Fig. 2-20(a) would look like if received by a SPARC, which numbers its bytes with byte 0 at the left (high-order byte) instead of at the right (low-order byte) as do all the Intel chips. When the server stub reads the parameters at addresses 0 and 4, respectively, it will find an integer equal to 83,886,080 (5×2^{24}) and a string "JILL".

One obvious, but unfortunately incorrect, approach is to invert the bytes of each word after they are received, leading to Fig. 2-20(c). Now the integer is 5 and the string is "LLIJ". The problem here is that integers are reversed by the different byte ordering, but strings are not. Without additional information about what is a string and what is an integer, there is no way to repair the damage.

Fortunately, this information is implicitly available. Remember that the items in the message correspond to the procedure identifier and parameters. Both the client and server know what the types of the parameters are. Thus a message corresponding to a remote procedure with n parameters will have $n + 1$ fields, one identifying the procedure and one for each of the n parameters. Once a standard has been agreed upon for representing each of the basic data types, given a parameter list and a message, it is possible to deduce which bytes belong to which parameter, and thus to solve the problem.

As a simple example, consider the procedure of Fig. 2-21(a). It has three parameters, a character, a floating-point number, and an array of five integers. We might decide to transmit a character in the rightmost byte of a word (leaving

the next 3 bytes empty), a float as a whole word, and an array as a group of words equal to the array length, preceded by a word giving the length, as shown in Fig. 2-21(b). Thus given these rules, the client stub for *foobar* knows that it must use the format of Fig. 2-21(b), and the server stub knows that incoming messages for *foobar* will have the format of Fig. 2-21(b). Having the type information for the parameters makes it possible to make any necessary conversions.

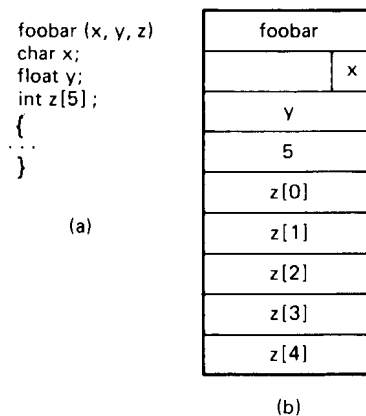


Fig. 2-21. (a) A procedure. (b) The corresponding message.

Even with this additional information, there are still some issues open. In particular, how should information be represented in the messages? One way is to devise a network standard or **canonical form** for integers, characters, Booleans, floating-point numbers, and so on, and require all senders to convert their internal representation to this form while marshaling. For example, suppose that it is decided to use two's complement for integers, ASCII for characters, 0 (false) and 1 (true) for Booleans, and IEEE format for floating-point numbers, with everything stored in little endian. For any list of integers, characters, Booleans, and floating-point numbers, the exact pattern required is now deterministic down to the last bit. As a result, the server stub no longer has to worry about which byte ordering the client has because the order of the bits in the message is now fixed, independent of the client's hardware.

The problem with this method is that it is sometimes inefficient. Suppose that a big endian client is talking to a big endian server. According to the rules, the client must convert everything to little endian in the message, and the server must convert it back again when it arrives. Although this is unambiguous, it requires two conversions when in fact none were necessary. This observation gives rise to a second approach: the client uses its own native format and indicates in the first byte of the message which format this is. Thus a little endian client builds a little endian message and a big endian client builds a big endian

message. As soon as a message comes in, the server stub examines the first byte to see what the client is. If it is the same as the server, no conversion is needed. Otherwise, the server stub converts everything. Although we have only discussed converting from one endian to the other, conversions between one's and two's complement, EBCDIC to ASCII, and so on, can be handled in the same way. The trick is knowing what the message layout is and what the client is. Once these are known, the rest is easy (provided that everyone can convert from everyone else's format).

Now we come to the question of where the stub procedures come from. In many RPC-based systems, they are generated automatically. As we have seen, given a specification of the server procedure and the encoding rules, the message format is uniquely determined. Thus it is possible to have a compiler read the server specification and generate a client stub that packs its parameters into the officially approved message format. Similarly, the compiler can also produce a server stub that unpacks them and calls the server. Having both stub procedures generated from a single formal specification of the server not only makes life easier for the programmers, but reduces the chance of error and makes the system transparent with respect to differences in internal representation of data items.

Finally, we come to our last and most difficult problem: How are pointers passed? The answer is: only with the greatest of difficulty, if at all. Remember that a pointer is meaningful only within the address space of the process in which it is being used. Getting back to our *read* example discussed earlier, if the second parameter (the address of the buffer) happens to be 1000 on the client, one cannot just pass the number 1000 to the server and expect it to work. Address 1000 on the server might be in the middle of the program text.

One solution is just to forbid pointers and reference parameters in general. However, these are so important that this solution is highly undesirable. In fact, it is not necessary either. In the *read* example, the client stub knows that the second parameter points to an array of characters. Suppose, for the moment, that it also knows how big the array is. One strategy then becomes apparent: copy the array into the message and send it to the server. The server stub can then call the server with a pointer to this array, even though this pointer has a different numerical value than the second parameter of *read* has. Changes the server makes using the pointer (e.g., storing data into it) directly affect the message buffer inside the server stub. When the server finishes, the original message can be sent back to the client stub, which then copies it back to the client. In effect, call-by-reference has been replaced by copy/restore. Although this is not always identical, it frequently is good enough.

One optimization makes this mechanism twice as efficient. If the stubs know whether the buffer is an input parameter or an output parameter to the server, one of the copies can be eliminated. If the array is input to the server

(e.g., in a call to *write*) it need not be copied back. If it is output, it need not be sent over in the first place. The way to tell them is in the formal specification of the server procedure. Thus associated with every remote procedure is a formal specification of the procedure, written in some kind of specification language, telling what the parameters are, which are input and which are output (or both), and what their (maximum) sizes are. It is from this formal specification that the stubs are generated by a special stub compiler.

As a final comment, it is worth noting that although we can now handle pointers to simple arrays and structures, we still cannot handle the most general case of a pointer to an arbitrary data structure such as a complex graph. Some systems attempt to deal with this case by actually passing the pointer to the server stub and generating special code in the server procedure for using pointers.

Normally, a pointer is followed (dereferenced) by putting it in a register and indirecting through the register. When this special technique is used, a pointer is dereferenced by sending a message back to the client stub asking it to fetch and send the item being pointed to (reads) or store a value at the address pointed to (writes). While this method works, it is often highly inefficient. Imagine having the file server store the bytes in the buffer by sending back each one in a separate message. Still, it is better than nothing, and some systems use it.

2.4.3. Dynamic Binding

An issue that we have glossed over so far is how the client locates the server. One method is just to hardwire the network address of the server into the client. The trouble with this approach is that it is extremely inflexible. If the server moves or if the server is replicated or if the interface changes, numerous programs will have to be found and recompiled. To avoid all these problems, some distributed systems use what is called **dynamic binding** to match up clients and servers. In this section we will describe the ideas behind dynamic binding.

The starting point for dynamic binding is the server's formal specification. As an example, consider the server of Fig. 2-9(a), specified in Fig. 2-22. The specification tells the name of the server (*file_server*), the version number (3.1), and a list of procedures provided by the server (*read*, *write*, *create*, and *delete*).

For each procedure, the types of the parameters are given. Each parameter is specified as being an *in* parameter, an *out* parameter, or an *in out* parameter. The direction is relative to the server. An *in* parameter, such as the file name, *name*, is sent from the client to the server. This one is used to tell the server which file to read from, write to, create, or delete. Similarly, *bytes* tells the server how many bytes to transfer and *position* tells where in the file to begin

```

#include <header.h>

specification of file_server, version 3.1:

    long read(in char name[MAX_PATH], out char buf[BUF_SIZE],
              in long bytes, in long position);

    long write(in char name[MAX_PATH], in char buf[BUF_SIZE],
              in long bytes, in long position);

    int create(in char[MAX_PATH], in int mode);

    int delete(in char[MAX_PATH]);

end;

```

Fig. 2-22. A specification of the stateless server of Fig. 2-9.

reading or writing. An *out* parameter such as *buf* in *read*, is sent from the server to the client. *Buf* is the place where the file server puts the data that the client has requested. An *in out* parameter, of which there are none in this example, would be sent from the client to the server, modified there, and then sent back to the client (copy/restore). Copy/restore is typically used for pointer parameters in cases where the server both reads and modifies the data structure being pointed to. The directions are crucial, so the client stub knows which parameters to send to the server, and the server stub knows which ones to send back.

As we pointed out earlier, this particular example is a stateless server. For a UNIX-like server, one would have additional procedures *open* and *close*, and different parameters for *read* and *write*. The concept of RPC itself is neutral, permitting the system designers to build any kind of servers they desire.

The primary use of the formal specification of Fig. 2-22 is as input to the stub generator, which produces both the client stub and the server stub. Both are then put into the appropriate libraries. When a user (client) program calls any of the procedures defined by this specification, the corresponding client stub procedure is linked into its binary. Similarly, when the server is compiled, the server stubs are linked with it too.

When the server begins executing, the call to *initialize* outside the main loop [see Fig. 2-9(a)] **exports** the server interface. What this means is that the server sends a message to a program called a **binder**, to make its existence known. This process is referred to as **registering** the server. To register, the server gives the binder its name, its version number, a unique identifier, typically 32 bits long, and a **handle** used to locate it. The handle is system dependent, and might be an Ethernet address, an IP address, an X.500 address, a sparse process identifier, or something else. In addition, other information, for example,

concerning authentication, might also be supplied. A server can also deregister with the binder when it is no longer prepared to offer service. The binder interface is shown in Fig. 2-23.

Call	Input	Output
Register	Name, version, handle, unique id	
Deregister	Name, version, unique id	
Lookup	Name, version	Handle, unique id

Fig. 2-23. The binder interface.

Given this background, now consider how the client locates the server. When the client calls one of the remote procedures for the first time, say, *read*, the client stub sees that it is not yet bound to a server, so it sends a message to the binder asking to **import** version 3.1 of the *file_server* interface. The binder checks to see if one or more servers have already exported an interface with this name and version number. If no currently running server is willing to support this interface, the *read* call fails. By including the version number in the matching process, the binder can ensure that clients using obsolete interfaces will fail to locate a server rather than locate one and get unpredictable results due to incorrect parameters.

On the other hand, if a suitable server exists, the binder gives its handle and unique identifier to the client stub. The client stub uses the handle as the address to send the request message to. The message contains the parameters and the unique identifier, which the server's kernel uses to direct the incoming message to the correct server in the event that several servers are running on that machine.

This method of exporting and importing interfaces is highly flexible. For example, it can handle multiple servers that support the same interface. The binder can spread the clients randomly over the servers to even the load if it wants to. It can also poll the servers periodically, automatically deregistering any server that fails to respond, to achieve a degree of fault tolerance. Furthermore, it can also assist in authentication. A server could specify, for example, that it only wished to be used by a specific list of users, in which case the binder would refuse to tell users not on the list about it. The binder can also verify that both client and server are using the same version of the interface.

However, this form of dynamic binding also has its disadvantages. The extra overhead of exporting and importing interfaces costs time. Since many client processes are short lived and each process has to start all over again, the effect may be significant. Also, in a large distributed system, the binder may become a bottleneck, so multiple binders are needed. Consequently, whenever

an interface is registered or deregistered, a substantial number of messages will be needed to keep all the binders synchronized and up to date, creating even more overhead.

2.4.4. RPC Semantics in the Presence of Failures

The goal of RPC is to hide communication by making remote procedure calls look just like local ones. With a few exceptions, such as the inability to handle global variables and the subtle differences introduced by using copy/restore for pointer parameters instead of call-by-reference, so far we have come fairly close. Indeed, as long as both client and server are functioning perfectly, RPC does its job remarkably well. The problem comes in when errors occur. It is then that the differences between local and remote calls are not always easy to mask. In this section we will examine some of the possible errors and what can be done about them.

To structure our discussion, let us distinguish between five different classes of failures that can occur in RPC systems, as follows:

1. The client is unable to locate the server.
2. The request message from the client to the server is lost.
3. The reply message from the server to the client is lost.
4. The server crashes after receiving a request.
5. The client crashes after sending a request.

Each of these categories poses different problems and requires different solutions.

Client Cannot Locate the Server

To start with, it can happen that the client cannot locate a suitable server. The server might be down, for example. Alternatively, suppose that the client is compiled using a particular version of the client stub, and the binary is not used for a considerable period of time. In the meantime, the server evolves and a new version of the interface is installed and new stubs are generated and put into use. When the client is finally run, the binder will be unable to match it up with a server and will report failure. While this mechanism is used to protect the client from accidentally trying to talk to a server that may not agree with it in terms of what parameters are required or what it is supposed to do, the problem remains of how this failure should be dealt with.

With the server of Fig. 2-9(a), each of the procedures returns a value, with

the code `-1` conventionally used to indicate failure. For such procedures, just returning `-1` will clearly tell the caller that something is amiss. In UNIX, a global variable, `errno`, is also assigned a value indicating the error type. In such a system, adding a new error type “Cannot locate server” is simple.

The trouble is, this solution is not general enough. Consider the *sum* procedure of Fig. 2-19. Here `-1` is a perfectly legal value to be returned, for example, the result of adding 7 to `-8`. Another error-reporting mechanism is needed.

One possible candidate is to have the error raise an **exception**. In some languages (e.g., Ada), programmers can write special procedures that are invoked upon specific errors, such as division by zero. In C, signal handlers can be used for this purpose. In other words, we could define a new signal type *SIGNOSERVER*, and allow it to be handled in the same way as other signals.

This approach, too, has drawbacks. To start with, not every language has exceptions or signals. To name one, Pascal does not. Another point is that having to write an exception or signal handler destroys the transparency we have been trying to achieve. Suppose that you are a programmer and your boss tells you to write the *sum* procedure. You smile and tell her it will be written, tested, and documented in five minutes. Then she mentions that you also have to write an exception handler as well, just in case the procedure is not there today. At this point it is pretty hard to maintain the illusion that remote procedures are no different from local ones, since writing an exception handler for “Cannot locate server” would be a rather unusual request in a single-processor system.

Lost Request Messages

The second item on the list is dealing with lost request messages. This is the easiest one to deal with: just have the kernel start a timer when sending the request. If the timer expires before a reply or acknowledgement comes back, the kernel sends the message again. If the message was truly lost, the server will not be able to tell the difference between the retransmission and the original, and everything will work fine. Unless, of course, so many request messages are lost that the kernel gives up and falsely concludes that the server is down, in which case we are back to “Cannot locate server.”

Lost Reply Messages

Lost replies are considerably more difficult to deal with. The obvious solution is just to rely on the timer again. If no reply is forthcoming within a reasonable period, just send the request once more. The trouble with this solution is that the client’s kernel is not really sure why there was no answer. Did the request or reply get lost, or is the server merely slow? It may make a difference.

In particular, some operations can safely be repeated as often as necessary

with no damage being done. A request such as asking for the first 1024 bytes of a file has no side effects and can be executed as often as necessary without any harm being done. A request that has this property is said to be **idempotent**.

Now consider a request to a banking server asking to transfer a million dollars from one account to another. If the request arrives and is carried out, but the reply is lost, the client will not know this and will retransmit the message. The bank server will interpret this request as a new one, and will carry it out too. Two million dollars will be transferred. Heaven forbid that the reply is lost 10 times. Transferring money is not idempotent.

One way of solving this problem is to try to structure all requests in an idempotent way. In practice, however, many requests (e.g., transferring money) are inherently nonidempotent, so something else is needed. Another method is to have the client's kernel assign each request a sequence number. By having each server's kernel keep track of the most recently received sequence number from each client's kernel that is using it, the server's kernel can tell the difference between an original request and a retransmission and can refuse to carry out any request a second time. An additional safeguard is to have a bit in the message header that is used to distinguish initial requests from retransmissions (the idea being that it is always safe to perform an original request; retransmissions may require more care).

Server Crashes

The next failure on the list is a server crash. It too relates to idempotency, but unfortunately it cannot be solved using sequence numbers. The normal sequence of events at a server is shown in Fig. 2-24(a). A request arrives, is carried out, and a reply is sent. Now consider Fig. 2-24(b). A request arrives and is carried out, just as before, but the server crashes before it can send the reply. Finally, look at Fig. 2-24(c). Again a request arrives, but this time the server crashes before it can even be carried out.

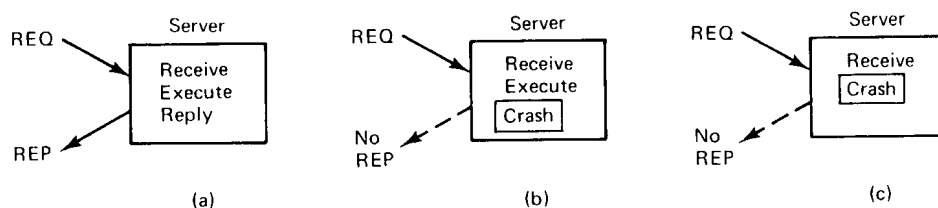


Fig. 2-24. (a) Normal case. (b) Crash after execution. (c) Crash before execution.

The annoying part of Fig. 2-24 is that the correct treatment differs for (b) and (c). In (b) the system has to report failure back to the client (e.g., raise an

exception), whereas in (c) it can just retransmit the request. The problem is that the client's kernel cannot tell which is which. All it knows is that its timer has expired.

Three schools of thought exist on what to do here. One philosophy is to wait until the server reboots (or rebinds to a new server) and try the operation again. The idea is to keep trying until a reply has been received, then give it to the client. This technique is called **at least once semantics** and guarantees that the RPC has been carried out at least one time, but possibly more.

The second philosophy gives up immediately and reports back failure. This way is called **at most once semantics** and guarantees that the RPC has been carried out at most one time, but possibly none at all.

The third philosophy is to guarantee nothing. When a server crashes, the client gets no help and no promises. The RPC may have been carried out anywhere from 0 to a large number of times. The main virtue of this scheme is that it is easy to implement.

None of these are terribly attractive. What one would like is **exactly once semantics**, but as can be seen fairly easily, there is no way to arrange this in general. Imagine that the remote operation consists of printing some text, and is accomplished by loading the printer buffer and then setting a single bit in some control register to start the printer. The crash can occur a microsecond before setting the bit, or a microsecond afterward. The recovery procedure depends entirely on which it is, but there is no way for the client to discover it.

In short, the possibility of server crashes radically changes the nature of RPC and clearly distinguishes single-processor systems from distributed systems. In the former case, a server crash also implies a client crash, so recovery is neither possible nor necessary. In the latter it is both possible and necessary to take some action.

Client Crashes

The final item on the list of failures is the client crash. What happens if a client sends a request to a server to do some work and crashes before the server replies? At this point a computation is active and no parent is waiting for the result. Such an unwanted computation is called an **orphan**.

Orphans can cause a variety of problems. As a bare minimum, they waste CPU cycles. They can also lock files or otherwise tie up valuable resources. Finally, if the client reboots and does the RPC again, but the reply from the orphan comes back immediately afterward, confusion can result.

What can be done about orphans? Nelson (1981) proposed four solutions. In solution 1, before a client stub sends an RPC message, it makes a log entry telling what it is about to do. The log is kept on disk or some other medium that

survives crashes. After a reboot, the log is checked and the orphan is explicitly killed off. This solution is called **extermination**.

The disadvantage of this scheme is the horrendous expense of writing a disk record for every RPC. Furthermore, it may not even work, since orphans themselves may do RPCs, thus creating **grandorphans** or further descendants that are impossible to locate. Finally, the network may be partitioned, due to a failed gateway, making it impossible to kill them, even if they can be located. All in all, this is not a promising approach.

In solution 2, called **reincarnation**, all these problems can be solved without the need to write disk records. The way it works is to divide time up into sequentially numbered epochs. When a client reboots, it broadcasts a message to all machines declaring the start of a new epoch. When such a broadcast comes in, all remote computations are killed. Of course, if the network is partitioned, some orphans may survive. However, when they report back, their replies will contain an obsolete epoch number, making them easy to detect.

Solution 3 is a variant on this idea, but less Draconian. It is called **gentle reincarnation**. When an epoch broadcast comes in, each machine checks to see if it has any remote computations, and if so, tries to locate their owner. Only if the owner cannot be found is the computation killed.

Finally, we have solution 4, **expiration**, in which each RPC is given a standard amount of time, T , to do the job. If it cannot finish, it must explicitly ask for another quantum, which is a nuisance. On the other hand, if after a crash the server waits a time T before rebooting, all orphans are sure to be gone. The problem to be solved here is choosing a reasonable value of T in the face of RPCs with wildly differing requirements.

In practice, none of these methods are desirable. Worse yet, killing an orphan may have unforeseen consequences. For example, suppose that an orphan has obtained locks on one or more files or data base records. If the orphan is suddenly killed, these locks may remain forever. Also, an orphan may have already made entries in various remote queues to start up other processes at some future time, so even killing the orphan may not remove all traces of it. Orphan elimination is discussed in more detail by Panzieri and Shrivastava (1988).

2.4.5. Implementation Issues

The success or failure of a distributed system often hinges on its performance. The system performance, in turn, is critically dependent on the speed of communication. The communication speed, more often than not, stands or falls with its implementation, rather than with its abstract principles. In this section we will look at some of the implementation issues for RPC systems, with a special emphasis on the performance and where the time is spent.

RPC Protocols

The first issue is the choice of the RPC protocol. Theoretically, any old protocol will do as long as it gets the bits from the client's kernel to the server's kernel, but practically there are several major decisions to be made here, and the choices made can have a major impact on the performance. The first decision is between a connection-oriented protocol and a connectionless protocol. With a connection-oriented protocol, at the time the client is bound to the server, a connection is established between them. All traffic, in both directions, uses this connection.

The advantage of having a connection is that communication becomes much easier. When a kernel sends a message, it does not have to worry about it getting lost, nor does it have to deal with acknowledgements. All that is handled at a lower level, by the software that supports the connection. When operating over a wide-area network, this advantage is often too strong to resist.

The disadvantage, especially over a LAN, is the performance loss. All that extra software gets in the way. Besides, the main advantage (no lost packets) is hardly needed on a LAN, since LANs are so reliable. As a consequence, most distributed operating systems that are intended for use in a single building or campus use connectionless protocols.

The second major choice is whether to use a standard general-purpose protocol or one specifically designed for RPC. Since there are no standards in this area, using a custom RPC protocol often means designing your own (or borrowing a friend's). System designers are split about evenly on this one.

Some distributed systems use IP (or UDP, which is built on IP) as the basic protocol. This choice has several things going for it:

1. The protocol is already designed, saving considerable work.
2. Many implementations are available, again saving work.
3. These packets can be sent and received by nearly all UNIX systems.
4. IP and UDP packets are supported by many existing networks.

In short, IP and UDP are easy to use and fit in well with existing UNIX systems and networks such as the Internet. This makes it straightforward to write clients and servers that run on UNIX systems, which certainly aids in getting code running quickly and in testing it.

As usual, the downside is the performance. IP was not designed as an end-user protocol. It was designed as a base upon which reliable TCP connections could be established over recalcitrant internetworks. For example, it can deal with gateways that fragment packets into little pieces so they can pass through networks with a tiny maximum packet size. Although this feature is never

needed in a LAN-based distributed system, the IP packet header fields dealing with fragmentation have to be filled in by the sender and verified by the receiver to make them legal IP packets. IP packets have in total 13 header fields, of which three are useful: the source and destination addresses and the packet length. The other 10 just come along for the ride, and one of them, the header checksum, is time consuming to compute. To make matters worse, UDP has another checksum, covering the data as well.

The alternative is to use a specialized RPC protocol that, unlike IP, does not attempt to deal with packets that have been bouncing around the network for a few minutes and then suddenly materialize out of thin air at an inconvenient moment. Of course, the protocol has to be invented, implemented, tested, and embedded in existing systems, so it is considerably more work. Furthermore, the rest of the world tends not to jump with joy at the birth of yet another new protocol. In the long run, the development and widespread acceptance of a high-performance RPC protocol is definitely the way to go, but we are not there yet.

One last protocol-related issue is packet and message length. Doing an RPC has a large, fixed overhead, independent of the amount of data sent. Thus reading a 64K file in a single 64K RPC is vastly more efficient than reading it in 64 1K RPCs. It is therefore important that the protocol and network allow large transmissions. Some RPC systems are limited to small sizes (e.g., Sun Microsystem's limit is 8K). In addition, many networks cannot handle large packets (Ethernet's limit is 1536 bytes), so a single RPC will have to be split over multiple packets, causing extra overhead.

Acknowledgements

When large RPCs have to be broken up into many small packets as just described, a new issue arises: Should individual packets be acknowledged or not? Suppose, for example, that a client wants to write a 4K block of data to a file server, but the system cannot handle packets larger than 1K. One strategy, known as a **stop-and-wait protocol**, is for the client to send packet 0 with the first 1K, then wait for an acknowledgement from the server, as illustrated in Fig. 2-25(b). Then the client sends the second 1K, waits for another acknowledgement, and so on.

The alternative, often called a **blast protocol**, is simply for the client to send all the packets as fast as it can. With this method, the server acknowledges the entire message when *all* the packets have been received, not one by one. The blast protocol is illustrated in Fig. 2-25(c).

These protocols have quite different properties. With stop-and-wait, if a packet is damaged or lost, the client fails to receive an acknowledgement on

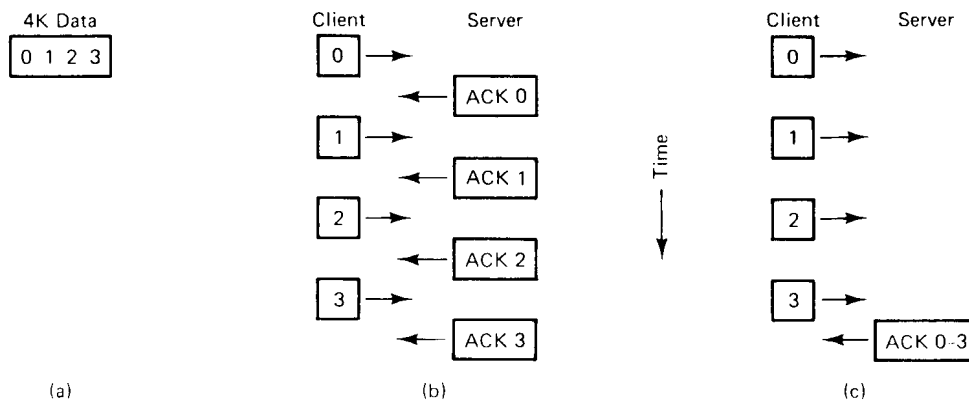


Fig. 2-25. (a) A 4K message. (b) A stop-and-wait protocol. (c) A blast protocol.

time, so it retransmits the one bad packet. With the blast protocol, the server is faced with a decision when, say, packet 1 is lost but packet 2 subsequently arrives correctly. It can abandon everything and do nothing, waiting for the client to time out and retransmit the entire message. Or alternatively, it can buffer packet 2 (along with 0), hope that 3 comes in correctly, and then specifically ask the client to send it packet 1. This technique is called **selective repeat**.

Both stop-and-wait and abandoning everything when an error occurs are easy to implement. Selective repeat requires more administration, but uses less network bandwidth. On highly reliable LANs, lost packets are so rare that selective repeat is usually more trouble than it is worth, but on wide-area networks it is frequently a good idea.

However, error control aside, there is another consideration that is actually more important: **flow control**. Many network interface chips are able to send consecutive packets with almost no gap between them, but they are not always able to receive an unlimited number of back-to-back packets due to finite buffer capacity on chip. With some designs, a chip cannot even accept two back-to-back packets because after receiving the first one, the chip is temporarily disabled during the packet-arrived interrupt, so it misses the start of the second one. When a packet arrives and the receiver is unable to accept it, an **overflow error** occurs and the incoming packet is lost. In practice, overflow errors are a much more serious problem than packets lost due to noise or other forms of damage.

The two approaches of Fig. 2-25 are quite different with respect to overflow errors. With stop-and-wait, overflow errors are impossible, because the second packet is not sent until the receiver has explicitly indicated that it is ready for it. (Of course, with multiple senders, overflow errors are still possible.)

With the blast protocol, receiver overflow is a possibility, which is

unfortunate, since the blast protocol is clearly much more efficient than stop-and-wait. However, there are also ways of dealing with overrun. If, on the one hand, the problem is caused by the chip being disabled temporarily while it is processing an interrupt, a smart sender can insert a delay between packets to give the receiver just enough time to generate the packet-arrived interrupt and reset itself. If the required delay is short, the sender can just loop (busy waiting); if it is long, it can set up a timer interrupt and go do something else while waiting. If it is in between (a few hundred microseconds), which it often is, probably the best solution is busy waiting and just accepting the wasted time as a necessary evil.

If, on the other hand, the overrun is caused by the finite buffer capacity of the network chip, say n packets, the sender can send n packets, followed by a substantial gap (or the protocol can be defined to require an acknowledgement after every n packets).

It should be clear that minimizing acknowledgement packets and getting good performance may be dependent on the timing properties of the network chip, so the protocol may have to be tuned to the hardware being used. A custom-designed RPC protocol can take issues like flow control into account more easily than a general-purpose protocol, which is why specialized RPC protocols usually outperform systems based on IP or UDP by a wide margin.

Before leaving the subject of acknowledgements, there is one other sticky point that is worth looking at. In Fig. 2-16(c) the protocol consists of a request, a reply, and an acknowledgement. The last one is needed to tell the server that it can discard the reply as it has arrived safely. Now suppose that the acknowledgement is lost in transit (unlikely, but not impossible). The server will not discard the reply. Worse yet, as far as the client is concerned, the protocol is finished. No timers are running and no packets are expected.

We could change the protocol to have acknowledgements themselves acknowledged, but this adds extra complexity and overhead for very little potential gain. In practice, the server can start a timer when sending the reply, and discard the reply when either the acknowledgement arrives or the timer expires. Also, a new request from the same client can be interpreted as a sign that the reply arrived, otherwise the client would not be issuing the next request.

Critical Path

Since the RPC code is so crucial to the performance of the system, let us take a closer look at what actually happens when a client performs an RPC with a remote server. The sequence of instructions that is executed on every RPC is called the **critical path**, and is depicted in Fig. 2-26. It starts when the client calls the client stub, proceeds through the trap to the kernel, the message

transmission, the interrupt on the server side, the server stub, and finally arrives at the server, which does the work and sends the reply back the other way.

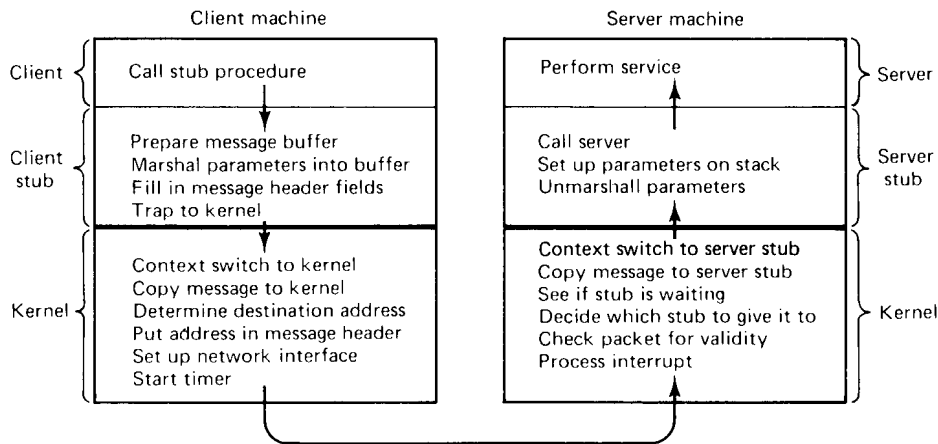


Fig. 2-26. Critical path from client to server.

Let us examine these steps a bit more carefully now. After the client stub has been called, its first job is to acquire a buffer into which it can assemble the outgoing message. In some systems, the client stub has a single fixed buffer that it fills in from scratch on every call. In other systems, a pool of partially filled in buffers is maintained, and an appropriate one for the server required is obtained. This method is especially appropriate when the underlying packet format has a substantial number of fields that must be filled in, but which do not change from call to call.

Next, the parameters are converted to the appropriate format and inserted into the message buffer, along with the rest of the header fields, if any. At this point the message is ready for transmission, so a trap to the kernel is issued.

When it gets control, the kernel switches context, saving the CPU registers and memory map, and setting up a new memory map that it will use while running in kernel mode. Since the kernel and user contexts are generally disjoint, the kernel must now explicitly copy the message into its address space so it can access it, fill in the destination address (and possibly other header fields), and have it copied to the network interface. At this point the client's critical path ends, as additional work done from here on does not add to the total RPC time: nothing the kernel does now affects how long it takes for the packet to arrive at the server. After starting the retransmission timer, the kernel can either enter a busy waiting loop to wait for the reply, or call the scheduler to look for another process to run. The former speeds up the processing of the reply, but effectively means that no multiprogramming can take place.

On the server side, the bits will come in and be put either in an on-board buffer or in memory by the receiving hardware. When all of them arrive, the receiver will generate an interrupt. The interrupt handler then examines the packet to see if it is valid, and determines which stub to give it to. If no stub is waiting for it, the handler must either buffer it or discard it. Assuming that a stub is waiting, the message is copied to the stub. Finally, a context switch is done, restoring the registers and memory map to the values they had at the time the stub called *receive*.

The server can now be restarted. It unmarshals the parameters and sets up an environment in which the server call be called. When everything is ready, the call is made. After the server has run, the path back to the client is similar to the forward path, but the other way.

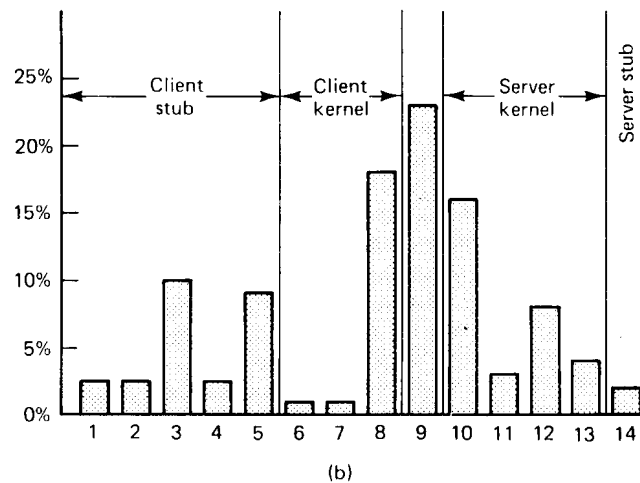
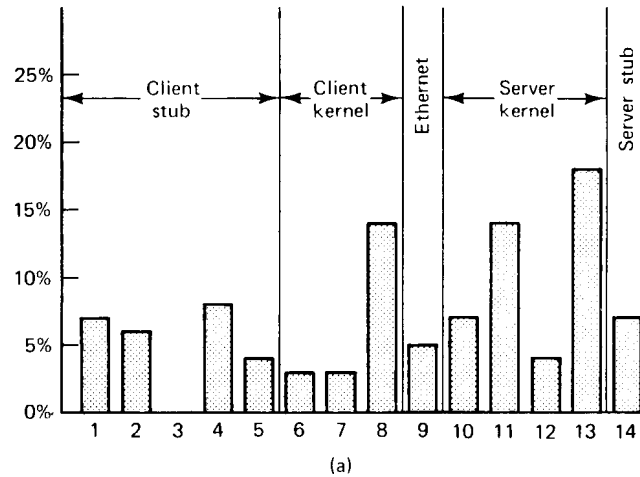
A question that all implementers are keenly interested in is: "Where is most of the time spent on the critical path?" Once that is known, work can begin on speeding it up. Schroeder and Burrows (1990) have provided us with a glimpse by analyzing in detail the critical path of the RPC on the DEC Firefly multiprocessor workstation. The results of their work are expressed in Fig. 2-27 as histograms with 14 bars, each bar corresponding to one of the steps from client to server (the reverse path is not shown, but is roughly analogous). Figure 2-27(a) gives results for a null RPC (no data), and Fig. 2-27(b) gives it for an array parameter with 1440 bytes. Although the fixed overhead is the same in both cases, considerably more time is needed for marshaling parameters and moving messages around in the second case.

For the null RPC, the dominant costs are the context switch to the server stub when a packet arrives, the interrupt service routine, and moving the packet to the network interface for transmission. For the 1440-byte RPC, the picture changes considerably, with the Ethernet transmission time now being the largest single component, with the time for moving the packet into and out of the interface coming in close behind.

Although Fig. 2-27 yields valuable insight into where the time is going, a few words of caution are necessary for interpreting these data. First, the Firefly is a multiprocessor, with five VAX CPUs. When the same measurements are run with only one CPU, the RPC time doubles, indicating that substantial parallel processing is taking place here, something that will not be true of most other machines.

Second, the Firefly uses UDP, and its operating system manages a pool of UDP buffers, which client stubs use to avoid having to fill in the entire UDP header every time.

Third, the kernel and user share the same address space, eliminating the need for context switches and for copying between kernel and user spaces, a great timesaver. Page table protection bits prevent the user from reading or writing parts of the kernel other than the shared buffers and certain other parts



- | | |
|----------------------------------|--|
| 1. Call stub | 8. Move packet to controller over the QBus |
| 2. Get message buffer | 9. Ethernet transmission time |
| 3. Marshal parameters | 10. Get packet from controller |
| 4. Fill in headers | 11. Interrupt service routine |
| 5. Compute UDP checksum | 12. Compute UDP checksum |
| 6. Trap to kernel | 13. Context switch to user space |
| 7. Queue packet for transmission | 14. Server stub code |

(c)

Fig. 2-27. Breakdown of the RPC critical path. (a) For a null RPC. (b) For an RPC with a 1440-byte array parameter. (c) The 14 steps in the RPC from client to server.

intended for user access. This design cleverly exploits particular features of the VAX architecture that facilitate sharing between kernel space and user space, but is not applicable to all computers.

Fourth and last, the entire RPC system has been carefully coded in assembly language and hand optimized. This last point is probably the reason that the various components in Fig. 2-27 are as uniform as they are. No doubt when the measurements were first made, they were more skewed, prompting the authors to attack the most time consuming parts until they no longer stuck out.

Schroeder and Burrows give some advice to future designers based on their experience. To start with, they recommend avoiding weird hardware (only one of the Firefly's five processors has access to the Ethernet, so packets have to be copied there before being sent, and getting them there is unpleasant). They also regret having based their system on UDP. The overhead, especially from the checksum, was not worth the cost. In retrospect, they believe a simple custom RPC protocol would have been better. Finally, using busy waiting instead of having the server stub go to sleep would have largely eliminated the single largest time sink in Fig. 2-27(a).

Copying

An issue that frequently dominates RPC execution times is copying. On the Firefly this effect does not show up because the buffers are mapped into both the kernel and user address spaces, but in most other systems the kernel and user address spaces are disjoint. The number of times a message must be copied varies from one to about eight, depending on the hardware, software, and type of call. In the best case, the network chip can DMA the message directly out of the client stub's address space onto the network (copy 1), depositing it in the server kernel's memory in real time (i.e., the packet-arrived interrupt occurs within a few microseconds of the last bit being DMA'ed out of the client stub's memory). Then the kernel inspects the packet and maps the page containing it into the server's address space. If this type of mapping is not possible, the kernel copies the packet to the server stub (copy 2).

In the worst case, the kernel copies the message from the client stub into a kernel buffer for subsequent transmission, either because it is not convenient to transmit directly from user space or the network is currently busy (copy 1). Later, the kernel copies the message, in software, to a hardware buffer on the network interface board (copy 2). At this point, the hardware is started, causing the packet to be moved over the network to the interface board on the destination machine (copy 3). When the packet-arrived interrupt occurs on the server's machine, the kernel copies it to a kernel buffer, probably because it cannot tell where to put it until it has examined it, which is not possible until it has extracted it from the hardware buffer (copy 4). Finally, the message has to be

copied to the server stub (copy 5). In addition, if the call has a large array passed as a value parameter, the array has to be copied onto the client's stack for the call stub, from the stack to the message buffer during marshaling within the client stub, and from the incoming message in the server stub to the server's stack preceding the call to the server, for three more copies, or eight in all.

Suppose that it takes an average of 500 nsec to copy a 32-bit word; then with eight copies, each word needs 4 microsec, giving a maximum data rate of about 1 Mbyte/sec, no matter how fast the network itself is. In practice, achieving even 1/10 of this would be pretty good.

One hardware feature that greatly helps eliminate unnecessary copying is **scatter-gather**. A network chip that can do scatter-gather can be set up to assemble a packet by concatenating two or more memory buffers. The advantage of this method is that the kernel can build the packet header in kernel space, leaving the user data in the client stub, with the hardware pulling them together as the packet goes out the door. Being able to gather up a packet from multiple sources eliminates copying. Similarly, being able to scatter the header and body of an incoming packet into different buffers also helps on the receiving end.

In general, eliminating copying is easier on the sending side than on the receiving side. With cooperative hardware, a reusable packet header inside the kernel and a data buffer in user space can be put out onto the network with no internal copying on the sending side. When it comes in at the receiver, however, even a very intelligent network chip will not know which server it should be given to, so the best the hardware can do is dump it into a kernel buffer and let the kernel figure out what to do with it.

In operating systems using virtual memory, a trick is available to avoid the copy to the stub. If the kernel packet buffer happens to occupy an entire page, beginning on a page boundary, and the server stub's receive buffer also happens to be an entire page, also starting on a page boundary, the kernel can change the memory map to map the packet buffer into the server's address space, simultaneously giving the server stub's buffer to the kernel. When the server stub starts running, its buffer will contain the packet, and this will have been achieved without copying.

Whether going to all this trouble is a good idea is a close call. Again assuming that it takes 500 nsec to copy a 32-bit word, copying a 1K packet takes 128 microsec. If the memory map can be updated in less time, mapping is faster than copying, otherwise it is not. This method also requires careful buffer control, making sure that all buffers are aligned properly with respect to page boundaries. If a buffer starts at a page boundary, the user process gets to see the entire packet, including the low-level headers, something that most systems try to hide in the name of portability.

Alternatively, if the buffers are aligned so that the header is at the end of one page and the data are at the start of the next, the data can be mapped without the

header. This approach is cleaner and more portable, but costs two pages per buffer: one mostly empty except for a few bytes of header at the end, and one for the data.

Finally, many packets are only a few hundred bytes, in which case it is doubtful that mapping will beat copying. Still, it is an interesting idea that is certainly worth thinking about.

Timer Management

All protocols consist of exchanging messages over some communication medium. In virtually all systems, messages can occasionally be lost, due either to noise or receiver overrun. Consequently, most protocols set a timer whenever a message is sent and an answer (reply or acknowledgement) is expected. If the reply is not forthcoming within the expected time, the timer expires and the original message is retransmitted. This process is repeated until the sender gets bored and gives up.

The amount of machine time that goes into managing the timers should not be underestimated. Setting a timer requires building a data structure specifying when the timer is to expire and what is to be done when that happens. The data structure is then inserted into a list consisting of the other pending timers. Usually, the list is kept sorted on time, with the next timeout at the head of the list and the most distant one at the end, as shown in Fig. 2-28.

When an acknowledgement or reply arrives before the timer expires, the timeout entry must be located and removed from the list. In practice, very few timers actually expire, so most of the work of entering and removing a timer from the list is wasted effort. Furthermore, timers need not be especially accurate. The timeout value chosen is usually a wild guess in the first place ("a few seconds sounds about right"). Besides, using a poor value does not affect the correctness of the protocol, only the performance. Too low a value will cause timers to expire too often, resulting in unnecessary retransmissions. Too high a value will cause a needlessly long delay in the event that a packet is actually lost.

The combination of these factors suggests that a different way of handling the timers might be more efficient. Most systems maintain a process table, with one entry containing all the information about each process in the system. While an RPC is being carried out, the kernel has a pointer to the current process table entry in a local variable. Instead of storing timeouts in a sorted linked list, each process table entry has a field for holding its timeout, if any, as shown in Fig. 2-28(b). Setting a timer for an RPC now consists of adding the length of the timeout to the current time and storing in the process table. Turning a timer off consists of merely storing a zero in the timer field. Thus the actions of setting and clearing timers are now reduced to a few machine instructions each.

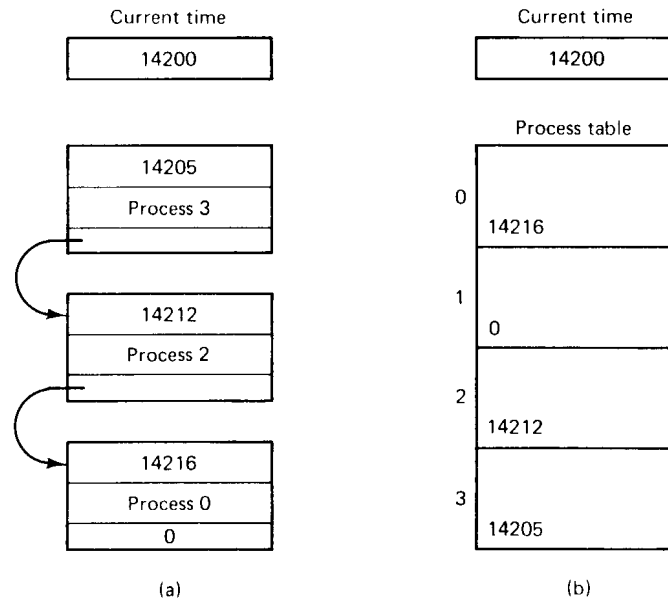


Fig. 2-28. (a) Timeouts in a sorted list. (b) Timeouts in the process table.

To make this method work, periodically (say, once per second), the kernel scans the entire process table, checking each timer value against the current time. Any nonzero value that is less than or equal to the current time corresponds to an expired timer, which is then processed and reset. For a system that sends, for example, 100 packets/sec, the work of scanning the process table once per second is only a fraction of the work of searching and updating a linked list 200 times a second. Algorithms that operate by periodically making a sequential pass through a table like this are called **sweep algorithms**.

2.4.6. Problem Areas

Remote procedure call using the client-server model is widely used as the basis for distributed operating systems. It is a simple abstraction that makes dealing with the complexity inherent in a distributed system more manageable than pure message passing. Nevertheless, there are a few problem areas that still have to be resolved. In this section we will discuss some of them.

Ideally, RPC should be transparent. That is, the programmer should not have to know which library procedures are local and which are remote. He should also be able to write procedures without regard to whether they will be

executed locally or remote. Even stricter, the introduction of RPC into a system that was previously run on a single CPU should not be accompanied by a set of new rules prohibiting constructions that were previously legal, or requiring constructions that were previously optional. Under this stringent criterion, few, if any, current distributed systems can be said to be completely transparent. Thus the holy grail of transparency will remain a research topic for the foreseeable future.

As an example, consider the problem of global variables. In single CPU systems these are legal, even for library procedures. For example, in UNIX, there is a global variable *errno*. After an incorrect system call, *errno* contains a code telling what went wrong. The existence of *errno* is public information, since the official UNIX standard, POSIX, requires it to be visible in one of the mandatory header files, *errno.h*. Thus it is not permitted for an implementation to hide it from the programmers.

Now suppose that a programmer writes two procedures that both directly access *errno*. One of these is run locally; the other is run remote. Since the compiler does not (and may not) know which variables and procedures are located where, no matter where *errno* is stored, one of the procedures will fail to access it correctly. The problem is that allowing local procedures unconstrained access to remote global variables, and vice versa, cannot be implemented, yet prohibiting this access violates the transparency principle (that programs should not have to act differently due to RPC).

A second problem is weakly-typed languages, like C. In a strongly-typed language, like Pascal, the compiler, and thus the stub procedure, knows everything there is to know about all the parameters. This knowledge allows the stub to marshal the parameters without difficulty. In C, however, it is perfectly legal to write a procedure that computes the inner product of two vectors (arrays), without specifying how large either one is. Each could be terminated by a special value known only to the calling and called procedure. Under these circumstances, it is essentially impossible for the client stub to marshal the parameters: it has no way of determining how large they are.

The usual solution is to force the programmer to define the maximum size when writing the formal definition of the server, but suppose that the programmer wants the procedure to work with any size input? He can put an arbitrary limit in the specification, say, 1 million, but that means that the client stub will have to pass 1 million elements even when the actual array size is 100 elements. Furthermore, the call will fail when the actual array is 1,000,001 elements or the total memory can only hold 200,000 elements.

A similar problem occurs when passing a pointer to a complex graph as a parameter. On a single CPU system, doing so works fine, but with RPC, the client stub has no way to find the entire graph.

Still another problem occurs because it is not always possible to deduce the

types of the parameters, not even from a formal specification or the code itself. An example is *printf*, which may have any number of parameters (at least one), and they can be an arbitrary mixture of integers, shorts, longs, characters, strings, floating point numbers of various lengths, and other types. Trying to call *printf* as a remote procedure would be practically impossible because C is so permissive. However, a rule saying that RPC can be used provided that you do not program in C would violate transparency.

The problems described above deal with transparency, but there is another class of difficulties that is even more fundamental. Consider the implementation of the UNIX command

```
sort <f1 >f2
```

Since *sort* knows it is reading standard input and writing standard output, it can act as a client for both input and output, performing RPCs with the file server to read *f1* as well as performing RPCs with the file server to write *f2*. Similarly, in the command

```
grep rat <f3 >f4
```

the *grep* program acts as a client to read the file *f3*, extracting only those lines containing the string “rat” and writing them to *f4*.

Now consider the UNIX pipeline

```
grep rat < f5 | sort >f6 .
```

As we have just seen, both *grep* and *sort* act as a client for both standard input and standard output. This behavior has to be compiled into the code to make the first two examples work. But how do they interact? Does *grep* act as a client doing writes to the server *sort*, or does *sort* act as the client doing reads from the server *grep*? Either way, one of them has to act as a server (i.e., passive), but as we have just seen, both have been programmed as clients (active). The difficulty here is that the client-server model really is not suitable at all.

In general, there is a problem with all pipelines of the form

```
p1 <f1 | p2 | p3 > f2
```

One approach to avoiding the client-client interface we just saw is to make the entire pipeline **read driven**, as illustrated in Fig. 2-29(b). The program *p1* acts as the (active) client and issues a read request to the file server to get *f1*. The program *p2*, also acting as a client, issues a read request to *p1* and the program *p3* issues a read request to *p2*. So far, so good. The trouble is that the file server does not act as a client issuing read requests to *p3* to collect the final output. Thus a read-driven pipeline does not work.

In Fig. 2-29(c) we see the write-driven approach. It has the mirror-image problem. Here *p1* acts as a client, doing writes to *p2*, which also acts as a client,

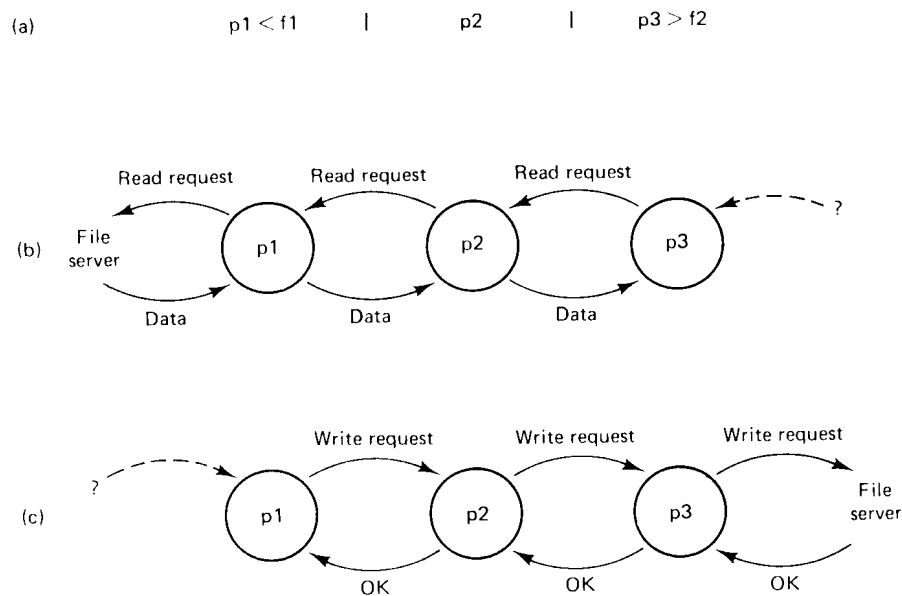


Fig. 2-29. (a) A pipeline. (b) The read-driven approach. (c) The write-driven approach.

doing writes to $p3$, which also acts as a client, writing to the file server, but there is no client issuing calls to $p1$ asking it to accept the input file.

While ad hoc solutions can be found, it should be clear that the client-server model inherent in RPC is not a good fit to this kind of communication pattern. As an aside, one possible ad hoc solution is to implement pipes as dual servers, responding to both write requests from the left and read requests from the right. Alternatively, pipes can be implemented with temporary files that are always read from, or written to, the file server. Doing so generates unnecessary overhead, however.

A similar problem occurs when the shell wants to get input from the user. Normally, it sends read requests to the terminal server, which simply collects keystrokes and waits until the shell asks for them. But what happens when the user hits the interrupt key (DEL, CTRL-C, break, etc.)? If the terminal server just passively puts the interrupt character in the buffer waiting until the shell asks for it, it will be impossible for the user to break off the current program. On the other hand, how can the terminal server act as a client and make an RPC to the shell, which is not expecting to act as a server? Clearly, this role reversal causes trouble, just as the role ambiguity does in the pipeline. In fact, any time an unexpected message has to be sent, there is a potential problem. While the client-server model is frequently a good fit, it is not perfect.

2.5. GROUP COMMUNICATION

An underlying assumption intrinsic to RPC is that communication involves only *two* parties, the client and the server. Sometimes there are circumstances in which communication involves multiple processes, not just two. For example, consider a group of file servers cooperating to offer a single, fault-tolerant file service. In such a system, it might be desirable for a client to send a message to all the servers, to make sure that the request could be carried out even if one of them crashed. RPC cannot handle communication from one sender to many receivers, other than by performing separate RPCs with each one. In this section we will discuss alternative communication mechanisms in which a message can be sent to multiple receivers in one operation.

2.5.1. Introduction to Group Communication

A group is a collection of processes that act together in some system or user-specified way. The key property that all groups have is that when a message is sent to the group itself, all members of the group receive it. It is a form of **one-to-many** communication (one sender, many receivers), and is contrasted with **point-to-point** communication in Fig. 2-30.

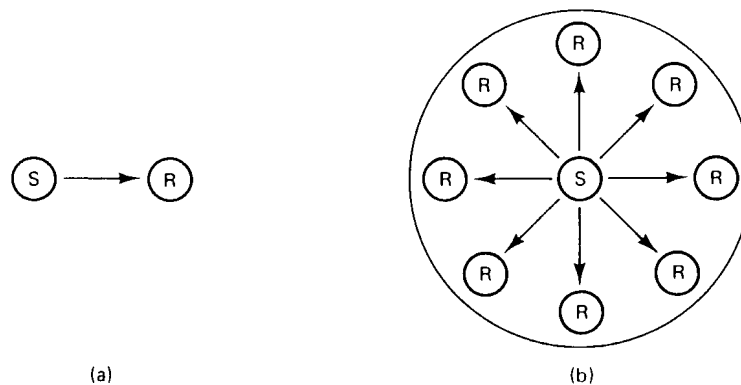


Fig. 2-30. (a) Point-to-point communication is from one sender to one receiver. (b) One-to-many communication is from one sender to multiple receivers.

Groups are dynamic. New groups can be created and old groups can be destroyed. A process can join a group or leave one. A process can be a member of several groups at the same time. Consequently, mechanisms are needed for managing groups and group membership.

Groups are roughly analogous to social organizations. A person might be a member of a book club, a tennis club, and an environmental organization. On a particular day, he might receive mailings (messages) announcing a new birthday cake cookbook from the book club, the annual Mother's Day tennis tournament from the tennis club, and the start of a campaign to save the Southern groundhog from the environmental organization. At any moment, he is free to leave any or all of these groups, and possibly join other groups.

Although in this book we will study only operating system (i.e., process) groups, it is worth mentioning that other groups are also commonly encountered in computer systems. For example, on the USENET computer network, there are hundreds of news groups, each about a specific subject. When a person sends a message to a particular news group, all members of the group receive it, even if there are tens of thousands of them. These higher-level groups usually have looser rules about who is a member, what the exact semantics of message delivery are, and so on, than do operating system groups. In most cases, this looseness is not a problem.

The purpose of introducing groups is to allow processes to deal with collections of processes as a single abstraction. Thus a process can send a message to a group of servers without having to know how many there are or where they are, which may change from one call to the next.

How group communication is implemented depends to a large extent on the hardware. On some networks, it is possible to create a special network address (for example, indicated by setting one of the high-order bits to 1), to which multiple machines can listen. When a packet is sent to one of these addresses, it is automatically delivered to all machines listening to the address. This technique is called **multicasting**. Implementing groups using multicast is straightforward: just assign each group a different multicast address.

Networks that do not have multicasting sometimes still have **broadcasting**, which means that packets containing a certain address (e.g., 0) are delivered to all machines. Broadcasting can also be used to implement groups, but it is less efficient. Each machine receives each broadcast, so its software must check to see if the packet is intended for it. If not, the packet is discarded, but some time is wasted processing the interrupt. Nevertheless, it still takes only one packet to reach all the members of a group.

Finally, if neither multicasting nor broadcasting is available, group communication can still be implemented by having the sender transmit separate packets to each of the members of the group. For a group with n members, n packets are required, instead of one packet when either multicasting or broadcasting is used. Although less efficient, this implementation is still workable, especially if most groups are small. The sending of a message from a single sender to a single receiver is sometimes called **unicasting** (point-to-point transmission), to distinguish it from multicasting and broadcasting.

2.5.2. Design Issues

Group communication has many of the same design possibilities as regular message passing, such as buffered versus unbuffered, blocking versus nonblocking, and so forth. However, there are also a large number of additional choices that must be made because sending to a group is inherently different from sending to a single process. Furthermore, groups can be organized in various ways internally. They can also be addressed in novel ways not relevant in point-to-point communication. In this section we will look at some of the most important design issues and point out the various alternatives.

Closed Groups versus Open Groups

Systems that support group communication can be divided into two categories depending on who can send to whom. Some systems support **closed groups**, in which only the members of the group can send to the group. Outsiders cannot send messages to the group as a whole, although they may be able to send messages to individual members. In contrast, other systems support **open groups**, which do not have this property. When open groups are used, any process in the system can send to any group. The difference between closed and open groups is shown in Fig. 2-31.

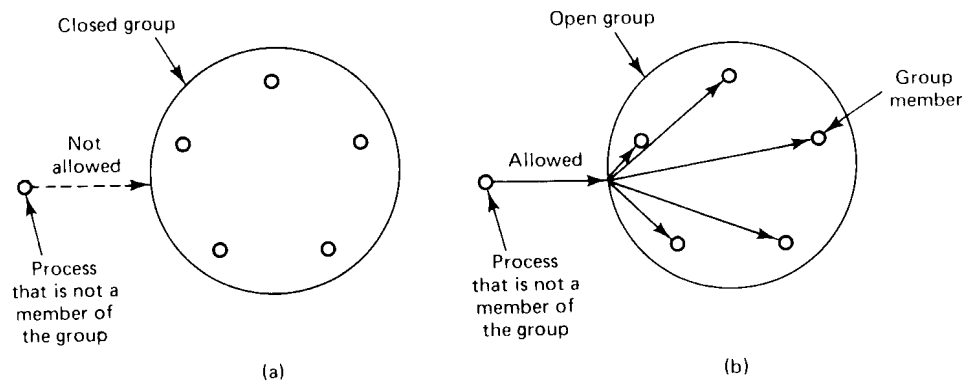


Fig. 2-31. (a) Outsiders may not send to a closed group. (b) Outsiders may send to an open group.

The decision as to whether a system supports closed or open groups usually relates to the reason groups are being supported in the first place. Closed groups are typically used for parallel processing. For example, a collection of processes working together to play a game of chess might form a closed group. They have their own goal and do not interact with the outside world.

On the other hand, when the idea of groups is to support replicated servers, it is important that processes that are not members (clients) can send to the group. In addition, the members of the group may also need to use group communication, for example to decide who should carry out a particular request. The distinction between closed and open groups is often made for implementation reasons.

Peer Groups versus Hierarchical Groups

The distinction between closed and open groups relates to who can communicate with the group. Another important distinction has to do with the internal structure of the group. In some groups, all the processes are equal. No one is boss and all decisions are made collectively. In other groups, some kind of hierarchy exists. For example, one process is the coordinator and all the others are workers. In this model, when a request for work is generated, either by an external client or by one of the workers, it is sent to the coordinator. The coordinator then decides which worker is best suited to carry it out, and forwards it there. More complex hierarchies are also possible, of course. These communication patterns are illustrated in Fig. 2-32.

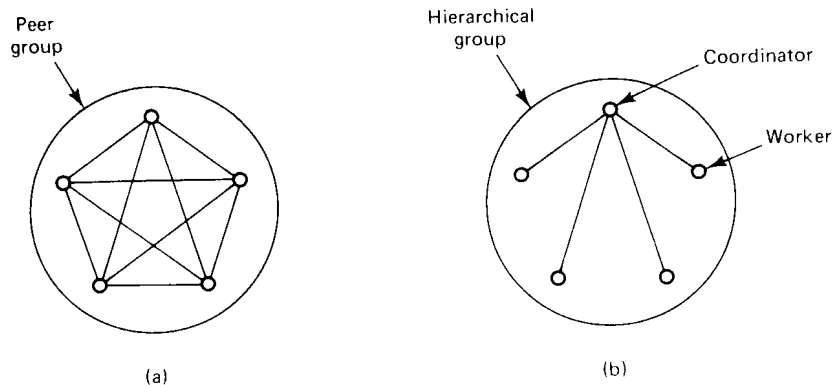


Fig. 2-32. (a) Communication in a peer group. (b) Communication in a simple hierarchical group.

Each of these organizations has its own advantages and disadvantages. The peer group is symmetric and has no single point of failure. If one of the processes crashes, the group simply becomes smaller, but can otherwise continue. A disadvantage is that decision making is more complicated. To decide anything, a vote has to be taken, incurring some delay and overhead.

The hierarchical group has the opposite properties. Loss of the coordinator brings the entire group to a grinding halt, but as long as it is running, it can make

decisions without bothering everyone else. For example, a hierarchical group might be appropriate for a parallel chess program. The coordinator takes the current board, generates all the legal moves from it, and farms them out to the workers for evaluation. During this evaluation, new boards are generated and sent back to the coordinator to have them evaluated. When a worker is idle, it asks the coordinator for a new board to work on. In this manner, the coordinator controls the search strategy and prunes the game tree (e.g., using the alpha-beta search method), but leaves the actual evaluation to the workers.

Group Membership

When group communication is present, some method is needed for creating and deleting groups, as well as for allowing processes to join and leave groups. One possible approach is to have a **group server** to which all these requests can be sent. The group server can then maintain a complete data base of all the groups and their exact membership. This method is straightforward, efficient, and easy to implement. Unfortunately, it shares with all centralized techniques a major disadvantage: a single point of failure. If the group server crashes, group management ceases to exist. Probably most or all groups will have to be reconstructed from scratch, possibly terminating whatever work was going on.

The opposite approach is to manage group membership in a distributed way. In an open group, an outsider can send a message to all group members announcing its presence. In a closed group, something similar is needed (in effect, even closed groups have to be open with respect to joining). To leave a group, a member just sends a goodbye message to everyone.

So far, all of this is straightforward. However, there are two issues associated with group membership that are a bit trickier. First, if a member crashes, it effectively leaves the group. The trouble is, there is no polite announcement of this fact as there is when a process leaves voluntarily. The other members have to discover this experimentally by noticing that the crashed member no longer responds to anything. Once it is certain that the crashed member is really down, it can be removed from the group.

The other knotty issue is that leaving and joining have to be synchronous with messages being sent. In other words, starting at the instant that a process has joined a group, it must receive all messages sent to that group. Similarly, as soon as a process has left a group, it must not receive any more messages from the group, and the other members must not receive any more messages from it. One way of making sure that a join or leave is integrated into the message stream at the right place is to convert this operation into a message sent to the whole group.

One final issue relating to group membership is what to do if so many machines go down that the group can no longer function at all. Some protocol is

needed to rebuild the group. Invariably, some process will have to take the initiative to start the ball rolling, but what happens if two or three try at the same time? The protocol will have to be able to withstand this.

Group Addressing

In order to send a message to a group, a process must have some way of specifying which group it means. In other words, groups need to be addressed, just as processes do. One way is to give each group a unique address, much like a process address. If the network supports multicast, the group address can be associated with a multicast address, so that every message sent to the group address can be multicast. In this way, the message will be sent to all those machines that need it, and no others.

If the hardware supports broadcast but not multicast, the message can be broadcast. Every kernel will then get it and extract from it the group address. If none of the processes on the machine is a member of the group, the broadcast is simply discarded. Otherwise, it is passed to all group members.

Finally, if neither multicast nor broadcast is supported, the kernel on the sending machine will have to have a list of machines that have processes belonging to the group. The kernel then sends each one a point-to-point message. These three implementation methods are shown in Fig. 2-33. The thing to notice is that in all three cases, a process just sends a message to a group address and it is delivered to all the members. How that happens is up to the operating system. The sender is not aware of the size of the group or whether communication is implemented by multicasting, broadcasting, or unicasting.

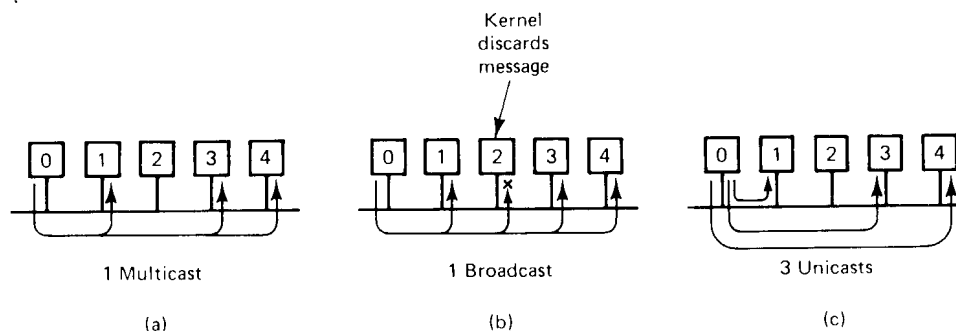


Fig. 2-33. Process 0 sending to a group consisting of processes 1, 3, and 4. (a) Multicast implementation. (b) Broadcast implementation. (c) Unicast implementation.

A second method of group addressing is to require the sender to provide an explicit list of all destinations (e.g., IP addresses). When this method is used,

the parameter in the call to *send* that specifies the destination is a pointer to a list of addresses. This method has the serious drawback that it forces user processes (i.e., the group members) to be aware of precisely who is a member of which group. In other words, it is not transparent. Furthermore, whenever group membership changes, the user processes must update their membership lists. In Fig. 2-33, this administration can easily be done by the kernels to hide it from the user processes.

Group communication also allows a third, and quite novel method of addressing as well, which we will call **predicate addressing**. With this system, each message is sent to all members of the group (or possibly the entire system) using one of the methods described above, but with a new twist. Each message contains a predicate (Boolean expression) to be evaluated. The predicate can involve the receiver's machine number, its local variables, or other factors. If the predicate evaluates to TRUE, the message is accepted. If it evaluates to FALSE, the message is discarded. Using this scheme it is possible, for example, to send a message to only those machines that have at least 4M of free memory and which are willing to take on a new process.

Send and Receive Primitives

Ideally, point-to-point and group communication should be merged into a single set of primitives. However, if RPC is the usual user communication mechanism, rather than raw *send* and *receive*, it is hard to merge RPC and group communication. Sending a message to a group cannot be modeled as a procedure call. The primary difficulty is that with RPC, the client sends one message to the server and gets back one answer. With group communication there are potentially n different replies. How can a procedure call deal with n replies? Consequently, a common approach is to abandon the (two-way) request/reply model underlying RPC and go back to explicit calls for sending and receiving (one-way model).

The library procedures that processes call to invoke group communication may be the same as for point-to-point communication or they may be different. If the system is based on RPC, user processes never call *send* and *receive* directly anyway, so there is less incentive to merge the point-to-point and group primitives. If user programs directly call *send* and *receive* themselves, there is something to be said for doing group communication with these existing primitives instead of inventing a new set.

Suppose, for the moment, that we wish to merge the two forms of communication. To send a message, one of the parameters of *send* indicates the destination. If it is a process address, a single message is sent to that one process. If it is a group address (or a pointer to a list of destinations), a message is sent to all members of the group. A second parameter to *send* points to the message.

The call can be buffered or unbuffered, blocking or nonblocking, reliable or not reliable, for both the point-to-point and group cases. Generally, these choices are made by the system designers and are fixed, rather than being selectable on a per message basis. Introducing group communication does not change this.

Similarly, *receive* indicates a willingness to accept a message, and possibly blocks until one is available. If the two forms of communication are merged, *receive* completes when either a point-to-point message or a group message arrives. However, since these two forms of communication are frequently used for different purposes, some systems introduce new library procedures, say, *group_send* and *group_receive*, so a process can indicate whether it wants a point-to-point or a group message.

In the design just described, communication is one-way. Replies are independent messages in their own right and are not associated with previous requests. Sometimes this association is desirable, to try to achieve more of the RPC flavor. In this case, after sending a message, a process is required to call *getreply* repeatedly to collect all the replies, one at a time.

Atomicity

A characteristic of group communication that we have alluded to several times is the all-or-nothing property. Most group communication systems are designed so that when a message is sent to a group, it will either arrive correctly at all members of the group, or at none of them. Situations in which some members receive a message and others do not are not permitted. The property of all-or-nothing delivery is called **atomicity** or **atomic broadcast**.

Atomicity is desirable because it makes programming distributed systems much easier. When any process sends a message to the group, it does not have to worry about what to do if some of them do not get it. For example, in a replicated distributed data base system, suppose that a process sends a message to all the data base machines to create a new record in the data base, and later sends a second message to update it. If some of the members miss the message creating the record, they will not be able to perform the update and the data base will become inconsistent. Life is just a lot simpler if the system guarantees that every message is delivered to all the members of the group, or if that is not possible, that it is not delivered to any, and that failure is reported back to the sender so it can take appropriate action to recover.

Implementing atomic broadcast is not quite as simple as it looks. The method of Fig. 2-33 fails because receiver overrun is possible at one or more machines. The only way to be sure that every destination receives every message is to require them to send back an acknowledgement upon message receipt. As long as machines never crash, this method will do.

However, many distributed systems aim at fault tolerance, so for them it is essential that atomicity also holds even in the presence of machine failures. In this light, all the methods of Fig. 2-33 are inadequate because some of the initial messages might not arrive due to receiver overrun, followed by the sender's crashing. Under these circumstances, some members of the group will have received the message and others will not have, precisely the situation that is unacceptable. Worse yet, the group members that have not received the message do not even know they are missing anything, so they cannot ask for a retransmission. Finally, with the sender now down, even if they did know, there is no one to provide the message.

Nevertheless, there is hope. Here is a simple algorithm that demonstrates that atomic broadcast is at least possible. The sender starts out by sending a message to all members of the group. Timers are set and retransmissions sent where necessary. When a process receives a message, if it has not yet seen this particular message, it, too, sends the message to all members of the group (again with timers and retransmissions if necessary). If it has already seen the message, this step is not necessary and the message is discarded. No matter how many machines crash or how many packets are lost, eventually all the surviving processes will get the message. Later we will describe more efficient algorithms for ensuring atomicity.

Message Ordering

To make group communication easy to understand and use, two properties are required. The first one is atomic broadcast, as discussed above. It ensures that a message sent to the group arrives at either all members or at none of them. The second property concerns message ordering. To see what the issue is here, consider Fig. 2-34, in which we have five machines, each with one process. Processes 0, 1, 3, and 4 belong to the same group. Processes 0 and 4 want to send a message to the group simultaneously. Assume that multicasting and broadcasting are not available, so that each process has to send three separate (unicast) messages. Process 0 sends to 1, 3, and 4; process 4 sends to 0, 1, and 3. These six messages are shown interleaved in time in Fig. 2-34(a).

The trouble is that when two processes are contending for access to a LAN, the order in which the messages are sent is nondeterministic. In Fig. 2-34(a) we see that (by accident), process 0 has won the first round and sends to process 1. Then process 4 wins three rounds in a row and sends to processes 0, 1, and 3. Finally, process 0 gets to send to 3 and 4. The order of these six messages is shown in different ways in the two parts of Fig. 2-34.

Now consider the situation as viewed by processes 1 and 3 as shown in Fig. 2-34(b). Process 1 first receives a message from 0, then immediately

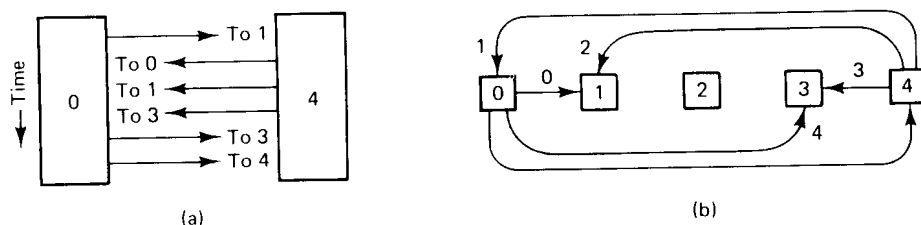


Fig. 2-34. (a) The three messages sent by processes 0 and 4 are interleaved in time. (b) Graphical representation of the six messages, showing the arrival order.

afterward it receives one from 4. Process 3 does not receive anything initially, then it receives messages from 4 and 0, in that order. Thus the two messages arrive in a different order. If processes 0 and 4 are both trying to update the same record in a data base, 1 and 3 end up with different final values. Needless to say, this situation is just as bad as one in which a (true hardware multicast) message sent to the group arrives at some members and not at others (atomicity failure). Thus to make programming reasonable, a system has to have well-defined semantics with respect to the order in which messages are delivered.

The best guarantee is to have all messages delivered instantaneously and in the order in which they were sent. If process 0 sends message *A* and then slightly later, process 4 sends message *B*, the system should first deliver *A* to all members of the group, and then deliver *B* to all members of the group. That way, all recipients get all messages in exactly the same order. This delivery pattern is something that programmers can understand and base their software on. We will call this **global time ordering**, since it delivers all messages in the exact order in which they were sent (conveniently ignoring the fact that according to Einstein's special theory of relativity there is no such thing as absolute global time).

Absolute time ordering is not always easy to implement, so some systems offer various watered-down variations. One of these is **consistent time ordering**, in which if two messages, say *A* and *B*, are sent close together in time, the system picks one of them as being "first" and delivers it to all group members, followed by the other. It may happen that the one chosen as first was not really first, but since no one knows this, the argument goes, system behavior should not depend on it. In effect, messages are guaranteed to arrive at all group members in the same order, but that order may not be the real order in which they were sent.

Even weaker time orderings have been used. We will study one of these, based on the idea of causality, when we come to ISIS later in this chapter.

Overlapping Groups

As we mentioned earlier, a process can be a member of multiple groups at the same time. This fact can lead to a new kind of inconsistency. To see the problem, look at Fig. 2-35, which shows two groups, 1 and 2. Processes *A*, *B*, and *C* are members of group 1. Processes *B*, *C*, and *D* are members of group 2.

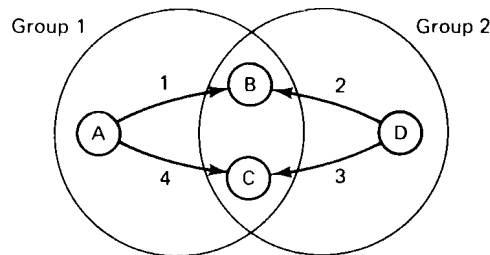


Fig. 2-35. Four processes, *A*, *B*, *C*, and *D*, and four messages. Processes *B* and *C* get the messages from *A* and *D* in a different order.

Now suppose that processes *A* and *D* each decide simultaneously to send a message to their respective groups, and that the system uses global time ordering within each group. As in our previous example, unicasting is used. The message order is shown in Fig. 2-35 by the numbers 1 through 4. Again we have the situation where two processes, in this case *B* and *C*, receive messages in a different order. *B* first gets a message from *A* followed by a message from *D*. *C* gets them in the opposite order.

The culprit here is that although there is a global time ordering within each group, there is not necessarily any coordination among multiple groups. Some systems support well-defined time ordering among overlapping groups and others do not. (If the groups are disjoint, the issue does not arise.) Implementing time ordering among different groups is frequently difficult to do, so the question arises as to whether it is worth it.

Scalability

Our final design issue is scalability. Many algorithms work fine as long as all the groups only have a few members, but what happens when there are tens, hundreds, or even thousands of members per group? Or thousands of groups? Also, what happens when the system is so large that it no longer fits on a single LAN, so multiple LANs and gateways are required? And what happens when the groups are spread over several continents?

The presence of gateways can affect many properties of the implementation. To start with, multicasting becomes more complicated. Consider, for example,

the internetwork shown in Fig. 2-36. It consists of four LANs and four gateways, to provide protection against the failure of any gateway.

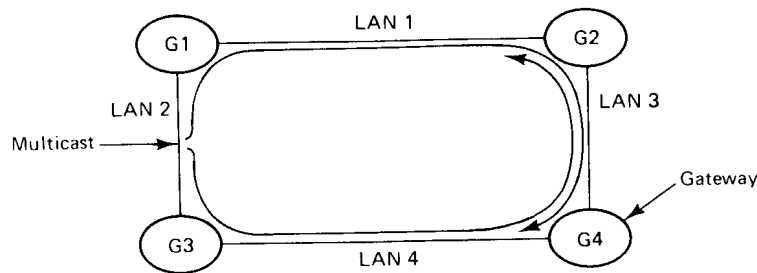


Fig. 2-36. Multicasting in an internetwork causes trouble.

Imagine that one of the machines on LAN 2 issues a multicast. When the multicast packet arrives at gateways *G1* and *G3*, what should they do? If they discard it, most of the machines will never see it, destroying its value as a multicast. If, however, the algorithm is just to have gateways forward all multicasts, then the packet will be copied to LAN 1 and LAN 4, and shortly thereafter to LAN 3 twice. Worse yet, gateway *G2* will see *G4*'s multicast and copy it to LAN 2, and vice versa. Clearly, a more sophisticated algorithm involving keeping track of previous packets is required to avoid exponential growth in the number of packets multicast.

Another problem with an internetwork is that some methods of group communication take advantage of the fact that only one packet can be on a LAN at any instant. In effect, the order of packet transmission defines an absolute global time order, which as we have seen, is frequently crucial. With gateways and multiple networks, it is possible for two packets to be "on the wire" simultaneously, thus destroying this useful property.

Finally, some algorithms may not scale well due to their computational complexity, their use of centralized components, or other factors.

2.5.3. Group Communication in ISIS

As an example of group communication, let us look at the ISIS system developed at Cornell (Birman, 1993; Birman and Joseph, 1987a, 1987b; and Birman and Van Renesse, 1994). ISIS is a toolkit for building distributed applications, for example, coordinating stock trading among all the brokers at a Wall Street securities firm. ISIS is not a complete operating system but rather, a set of programs that can run on top of UNIX or other existing operating systems. It is interesting to study because it has been widely described in the literature and

has been used for numerous real applications. In Chap. 7 we will study group communication in Amoeba, which takes a quite different approach.

The key idea in ISIS is **synchrony** and the key communication primitives are different forms of atomic broadcast. Before looking at how ISIS does atomic broadcast, it is necessary first to examine the various forms of synchrony it distinguishes. A **synchronous system** is one in which events happen strictly sequentially, with each event (e.g., a broadcast) taking essentially zero time to complete. For example, if process *A* sends a message to processes *B*, *C*, and *D*, as shown in Fig. 2-37(a), the message arrives instantaneously at all the destinations. Similarly, a subsequent message from *D* to the others also takes zero time to be delivered everywhere. As viewed by an outside observer, the system consists of discrete events, none of which ever overlap the others. This property makes it easy to understand system behavior.

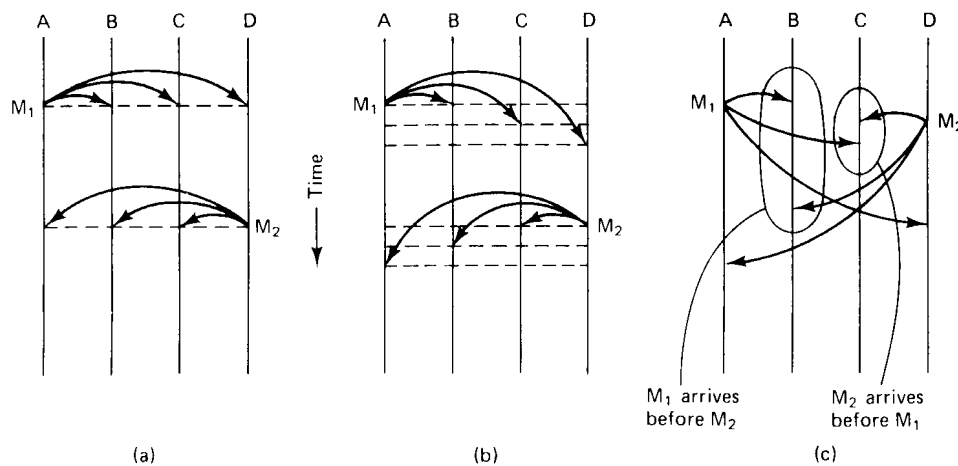


Fig. 2-37. (a) A synchronous system. (b) Loose synchrony. (c) Virtual synchrony.

Synchronous systems are impossible to build, so we need to investigate other types of systems, with weaker requirements on time. A **loosely synchronous system** is one like that of Fig. 2-37(b), in which events take a finite amount of time but all events appear in the same order to all parties. In particular, all processes receive all messages in the same order. Earlier, we discussed essentially the same idea under the name consistent time ordering.

Such systems are possible to build, but for some applications even weaker semantics are acceptable, and the hope is to be able to capitalize on these weak semantics to gain performance. Fig. 2-37(c) shows a **virtually synchronous system**, one in which the ordering constraint has been relaxed, but in such a way that under carefully selected circumstances, it does not matter.

Let us look at these circumstances. In a distributed system, two events are said to be **causally related** if the nature or behavior of the second one might have been influenced in any way by the first one. Thus if *A* sends a message to *B*, which inspects it and then sends a new message to *C*, the second message is causally related to the first one, since its contents might have been derived in part from the first one. Whether this actually happened is irrelevant. The relation holds if there *might* have been an influence.

Two events that are unrelated are said to be **concurrent**. If *A* sends a message to *B*, and about the same time, *C* sends a message to *D*, these events are concurrent because neither can influence the other. What virtual synchrony really means is that if two messages are causally related, all processes *must* receive them in the same (correct) order. If, however, they are concurrent, no guarantees are made, and the system is free to deliver them in a different order to different processes if this is easier. Thus when it matters, messages are always delivered in the same order, but when it does not matter, they may or may not be.

Communication Primitives in ISIS

Now we come to the broadcast primitives used in ISIS. Three of them have been defined: ABCAST, CBCAST, and GBCAST, all with different semantics. ABCAST provides loosely synchronous communication and is used for transmitting data to the members of a group. CBCAST provides virtually synchronous communication and is also used for sending data. GBCAST is somewhat like ABCAST, except that it is used for managing group membership rather than for sending ordinary data.

Originally, ABCAST used a form of two-phase commit protocol that worked like this. The sender, *A*, assigned a timestamp (actually just a sequence number) to the message and sent it to all the group members (by explicitly naming them all). Each one picked its own timestamp, larger than any other timestamp number it had sent or received, and sent it back to *A*. When all of these arrived, *A* chose the largest one and sent a *Commit* message to all the members again containing it. Committed messages were delivered to the application programs in order of the timestamps. It can be shown that this protocol guarantees that all messages will be delivered to all processes in the same order.

It can also be shown that this protocol is complex and expensive. For this reason, the ISIS designers invented the CBCAST primitive, which guarantees ordered delivery only for messages that are causally related. (The ABCAST protocol just described has subsequently been replaced, but even the new one is much slower than CBCAST.) The CBCAST protocol works as follows. If a group has n members, each process maintains a vector with n components, one per group member. The i th component of this vector is the number of the last

message received in sequence from process i . The vectors are managed by the runtime system, not the user processes themselves, and are initialized to zero, as shown at the top of Fig. 2-38.

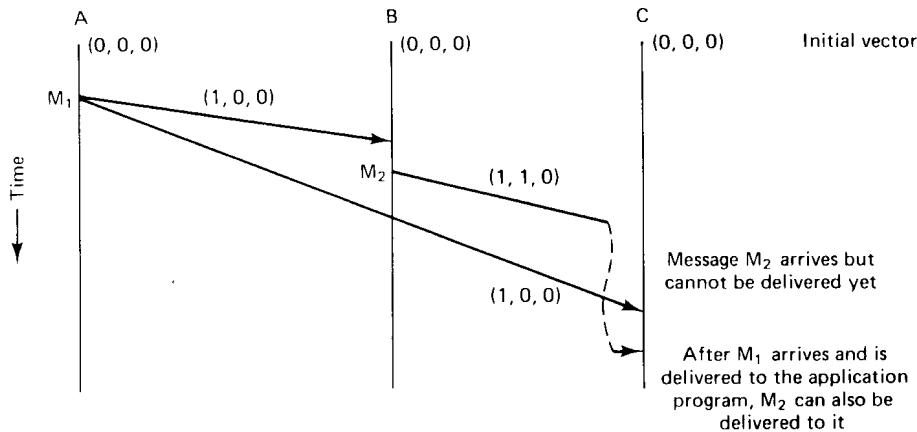


Fig. 2-38. Messages can be delivered only when all causally earlier messages have already been delivered.

When a process has a message to send, it increments its own slot in its vector, and sends the vector as part of the message. When M_1 in Fig. 2-38 gets to B , a check is made to see if it depends on anything that B has not yet seen. The first component of the vector is one higher than B 's own first component, which is expected (and required) for a message from A , and the others are the same, so the message is accepted and passed to the group member running on B . If any other component of the incoming vector had been larger than the corresponding component of B 's vector, the message could not have been delivered yet.

Now B sends a message of its own, M_2 , to C , which arrives before M_1 . From the vector, C sees that B had already received one message from A before M_2 was sent, and since it has not yet received anything from A , M_2 is buffered until a message from A arrives. Under no conditions may it be delivered before A 's message.

The general algorithm for deciding whether to pass an incoming message to the user process or delay it can now be stated. Let V_i be the i th component of the vector in the incoming message, and L_i be the i th component of the vector stored in the receiver's memory. Suppose that the message was sent by j . The first condition for acceptance is $V_j = L_j + 1$. This simply states that this is the next message in sequence from j , that is, no messages have been missed. (Messages from the same sender are always causally related.) The second condition for acceptance is $V_i \leq L_i$ for all $i \neq j$. This condition simply states that the

sender has not seen any message that the receiver has missed. If an incoming message passes both tests, the runtime system can pass it to the user process without delay. Otherwise, it must wait.

In Fig. 2-39 we show a more detailed example of the vector mechanism. Here process 0 has sent a message containing the vector (4, 6, 8, 2, 1, 5) to the other five members of its group. Process 1 has seen the same messages as process 0 except for message 7 just sent by process 1 itself, so the incoming message passes the test, is accepted, and can be passed up to the user process. Process 2 has missed message 6 sent by process 1, so the incoming message must be delayed. Process 3 has seen everything the sender has seen, and in addition message 7 from process 1, which apparently has not yet gotten to process 0, so the message is accepted. Process 4 missed the previous message from 0 itself. This omission is serious, so the new message will have to wait. Finally, process 5 is also slightly ahead of 0, so the message can be accepted immediately.

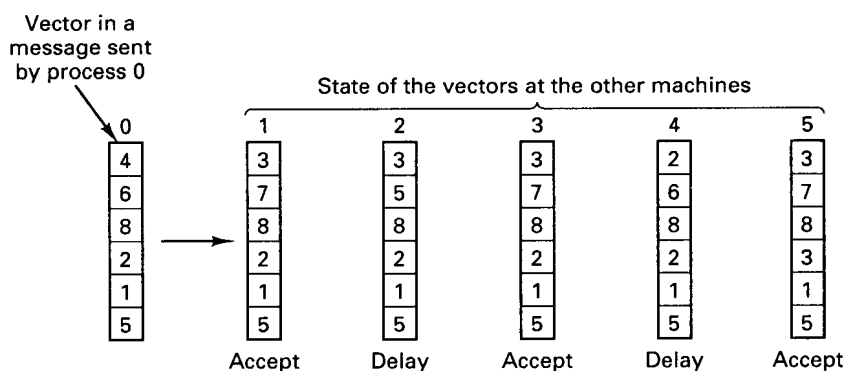


Fig. 2-39. Examples of the vectors used by CBCAST.

ISIS also provides fault tolerance and support for message ordering for overlapping groups using CBCAST. The algorithms used are somewhat complicated, though. For details, see (Birman et al., 1991).

2.6. SUMMARY

The key difference between a centralized operating system and a distributed one is the importance of communication in the latter. Various approaches to communication in distributed systems have been proposed and implemented. For relatively slow, wide-area distributed systems, connection-oriented layered protocols such as OSI and TCP/IP are sometimes used because the main problem to be overcome is how to transport the bits reliably over poor physical lines.

For LAN-based distributed systems, layered protocols are rarely used. Instead, a much simpler model is usually adopted, in which the client sends a message to the server and the server sends back a reply to the client. By eliminating most of the layers, much higher performance can be achieved. Many of the design issues in these message-passing systems concern the communication primitives: blocking versus nonblocking, buffered versus unbuffered, reliable versus unreliable, and so on.

The problem with the basic client-server model is that conceptually inter-process communication is handled as I/O. To present a better abstraction, remote procedure call is widely used. With RPC, a client running on one machine calls a procedure running on another machine. The runtime system, embodied in stub procedures, handles collecting parameters, building messages, and the interface with the kernel to actually move the bits.

Although RPC is a step forward above raw message passing, it has its own problems. The correct server has to be located. Pointers and complex data structures are hard to pass. Global variables are difficult to use. The exact semantics of RPC are tricky because clients and servers can fail independently of one another. Finally, implementing RPC efficiently is not straightforward and requires careful thought.

RPC is limited to those situations where a single client wants to talk to a single server. When a collection of processes, for example, replicated file servers, need to communicate with each other as a group, something else is needed. Systems such as ISIS provide a new abstraction for this purpose: group communication. ISIS offers a variety of primitives, the most important of which is CBCAST. CBCAST offers weakened communication semantics based on causality and implemented by including sequence number vectors in each message to allow the receiver to see whether the message should be delivered immediately or delayed until some prior messages have arrived.

PROBLEMS

1. In many layered protocols, each layer has its own header. Surely it would be more efficient to have a single header at the front of each message with all the control in it than all these separate headers. Why is this not done?
2. What is meant by an *open system*? Why are some systems not open?
3. What is the difference between a connection-oriented and connectionless communication protocol?

4. An ATM system is transmitting cells at the OC-3 rate. Each packet is 48 bytes long, and thus fits into a cell. An interrupt takes 1 μ sec. What fraction of the CPU is devoted to interrupt handling? Now repeat this problem for 1024-byte packets.
5. What is the probability that a totally garbled ATM header will be accepted as being correct?
6. Suggest a simple modification to Fig. 2-9 that reduces network traffic.
7. If the communication primitives in a client-server system are nonblocking, a call to *send* will complete before the message has actually been sent. To reduce overhead, some systems do not copy the data to the kernel, but transmit it directly from user space. For such a system, devise two ways in which the sender can be told that the transmission has been completed and the buffer can be reused.
8. In many communication systems, calls to *send* set a timer to guard against hanging the client forever if the server crashes. Suppose that a fault-tolerant system is implemented using multiple processors for all clients and all servers, so the probability of a client or server crashing is effectively zero. Do you think it is safe to get rid of timeouts in this system?
9. When buffered communication is used, a primitive is normally available for user processes to create mailboxes. In the text it was not specified whether this primitive must specify the size of the mailbox. Give an argument each way.
10. In all the examples in this chapter, a server can only listen to a single address. In practice, it is sometimes convenient for a server to listen to multiple addresses at the same time, for example, if the same process performs a set of closely related services that have been assigned separate addresses. Invent a scheme by which this goal can be accomplished.
11. Consider a procedure *incr* with two integer parameters. The procedure adds one to each parameter. Now suppose that it is called with the same variable twice, for example, as *incr(i, i)*. If *i* is initially 0, what value will it have afterward if call-by-reference is used? How about if copy/restore is used?
12. Pascal has a construction called a record variant, in which a field of a record can hold any one of several alternatives. At run time, there is no sure-fire way to tell which one is in there. Does this feature of Pascal have any implications for remote procedure call? Explain your answer.
13. The usual sequence of steps in an RPC involves trapping to the kernel to have the message sent from the client to the server. Suppose that a special

co-processor chip for doing network I/O exists and that this chip is directly addressable from user space. Would it be worth having? What steps would an RPC consist of in that case?

14. The SPARC chip uses a 32-bit word in big endian format. If a SPARC sends the integer 2 to a 486, which is little endian, what numerical value does the 486 see?
15. One way to handle parameter conversion in RPC systems is to have each machine send parameters in its native representation, with the other one doing the translation, if need be. In the text it was suggested that the native system could be indicated by a code in the first byte. However, since locating the first byte in the first word is precisely the problem, can this work, or is the book wrong?
16. In Fig. 2-23 the *deregister* call to the binder has the unique identifier as one of the parameters. Is this really necessary? After all, the name and version are also provided, which uniquely identifies the service.
17. Reading the first block of a file from a remote file server is an idempotent operation. What about writing the first block?
18. For each of the following applications, do you think at least once semantics or at most once semantics is best? Discuss.
 - (a) Reading and writing files from a file server.
 - (b) Compiling a program.
 - (c) Remote banking.
19. Suppose that the time to do a null RPC (i.e., 0 data bytes) is 1.0 msec, with an additional 1.5 msec for every 1K of data. How long does it take to read 32K from the file server in a single 32K RPC? How about as 32 1K RPCs?
20. How can atomic broadcast be used to manage group membership?
21. When a computation runs for a long time, it is sometimes wise to make checkpoints periodically, that is, to save the state of the process on disk in case it crashes. In that way, the process can be restarted from the checkpoint instead of from the beginning. Try to devise a way of checkpointing a computation that consists of multiple processes running in parallel.
22. Imagine that in a particular distributed system all the machines are redundant multiprocessors, so that the possibility of a machine crashing is so low that it can be ignored. Devise a simple method for implementing global time-ordered atomic broadcast using only unicasting. (*Hint*: Arrange the machines in a logical ring.)

3

Synchronization in Distributed Systems

In Chap. 2, we saw how processes in a distributed system communicate with one another. The methods used include layered protocols, request/reply message passing (including RPC), and group communication. While communication is important, it is not the entire story. Closely related is how processes cooperate and synchronize with one another. For example, how are critical regions implemented in a distributed system, and how are resources allocated? In this chapter we will study these and other issues related to interprocess cooperation and synchronization in distributed systems.

In single CPU systems, critical regions, mutual exclusion, and other synchronization problems are generally solved using methods such as semaphores and monitors. These methods are not well suited to use in distributed systems because they invariably rely (implicitly) on the existence of shared memory. For example, two processes that are interacting using a semaphore must both be able to access the semaphore. If they are running on the same machine, they can share the semaphore by having it stored in the kernel, and execute system calls to access it. If, however, they are running on different machines, this method no longer works, and other techniques are needed. Even seemingly simple matters, such as determining whether event *A* happened before or after event *B*, require careful thought.

We will start out by looking at time and how it can be measured, because time plays a major role in some synchronization methods. Then we will look at

mutual exclusion and election algorithms. After that we will study a high-level synchronization technique called atomic transactions. Finally, we will look at deadlock in distributed systems.

3.1. CLOCK SYNCHRONIZATION

Synchronization in distributed systems is more complicated than in centralized ones because the former have to use distributed algorithms. It is usually not possible (or desirable) to collect all the information about the system in one place, and then let some process examine it and make a decision as is done in the centralized case. In general, distributed algorithms have the following properties:

1. The relevant information is scattered among multiple machines.
2. Processes make decisions based only on local information.
3. A single point of failure in the system should be avoided.
4. No common clock or other precise global time source exists.

The first three points all say that it is unacceptable to collect all the information in a single place for processing. For example, to do resource allocation (assigning I/O devices in a deadlock-free way), it is generally not acceptable to send all the requests to a single manager process, which examines them all and grants or denies requests based on information in its tables. In a large system, such a solution puts a heavy burden on that one process.

Furthermore, having a single point of failure like this makes the system unreliable. Ideally, a distributed system should be more reliable than the individual machines. If one goes down, the rest should be able to continue to function. Having the failure of one machine (e.g., the resource allocator) bring a large number of other machines (its customers) to a grinding halt is the last thing we want. Achieving synchronization without centralization requires doing things in a different way from traditional operating systems.

The last point in the list is also crucial. In a centralized system, time is unambiguous. When a process wants to know the time, it makes a system call and the kernel tells it. If process *A* asks for the time, and then a little later process *B* asks for the time, the value that *B* gets will be higher than (or possibly equal to) the value *A* got. It will certainly not be lower. In a distributed system, achieving agreement on time is not trivial.

Just think, for a moment, about the implications of the lack of global time on the UNIX *make* program, as a single example. Normally, in UNIX, large programs are split up into multiple source files, so that a change to one source file

only requires one file to be recompiled, not all the files. If a program consists of 100 files, not having to recompile everything because one file has been changed greatly increases the speed at which programmers can work.

The way *make* normally works is simple. When the programmer has finished changing all the source files, he starts *make*, which examines the times at which all the source and object files were last modified. If the source file *input.c* has time 2151 and the corresponding object file *input.o* has time 2150, *make* knows that *input.c* has been changed since *input.o* was created, and thus *input.c* must be recompiled. On the other hand, if *output.c* has time 2144 and *output.o* has time 2145, no compilation is needed here. Thus *make* goes through all the source files to find out which ones need to be recompiled and calls the compiler to recompile them.

Now imagine what could happen in a distributed system in which there is no global agreement on time. Suppose that *output.o* has time 2144 as above, and shortly thereafter *output.c* is modified but is assigned time 2143 because the clock on its machine is slightly slow, as shown in Fig. 3-1. *Make* will not call the compiler. The resulting executable binary program will then contain a mixture of object files from the old sources and the new sources. It will probably not work, and the programmer will go crazy trying to understand what is wrong with the code.

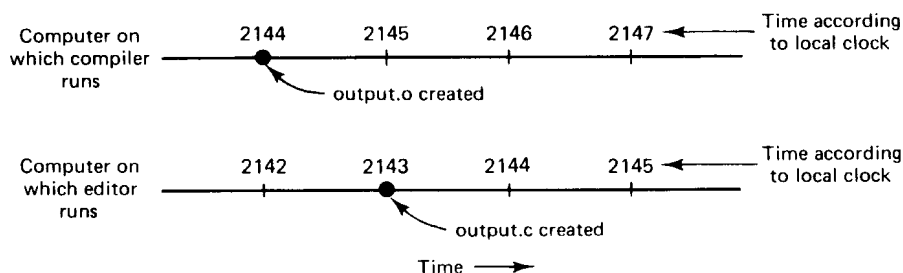


Fig. 3-1. When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

Since time is so basic to the way people think, and the effect of not having all the clocks synchronized can be so dramatic, as we have just seen, it is fitting that we begin our study of synchronization with the simple question: Is it possible to synchronize all the clocks in a distributed system?

3.1.1. Logical Clocks

Nearly all computers have a circuit for keeping track of time. Despite the widespread use of the word “clock” to refer to these devices, they are not actually clocks in the usual sense. **Timer** is perhaps a better word. A computer